



Fast Realistic Rendering

Physically-Based Rendering

Master in Innovation and Research in
Informàtics

Facultat d'Informàtica de Barcelona
Universitat Politècnica de Catalunya
- BarcelonaTech

Imanol Muñoz Pandiella
imanol.munoz@upc.edu

Realistic Rendering Goal



Matthias Jaworski

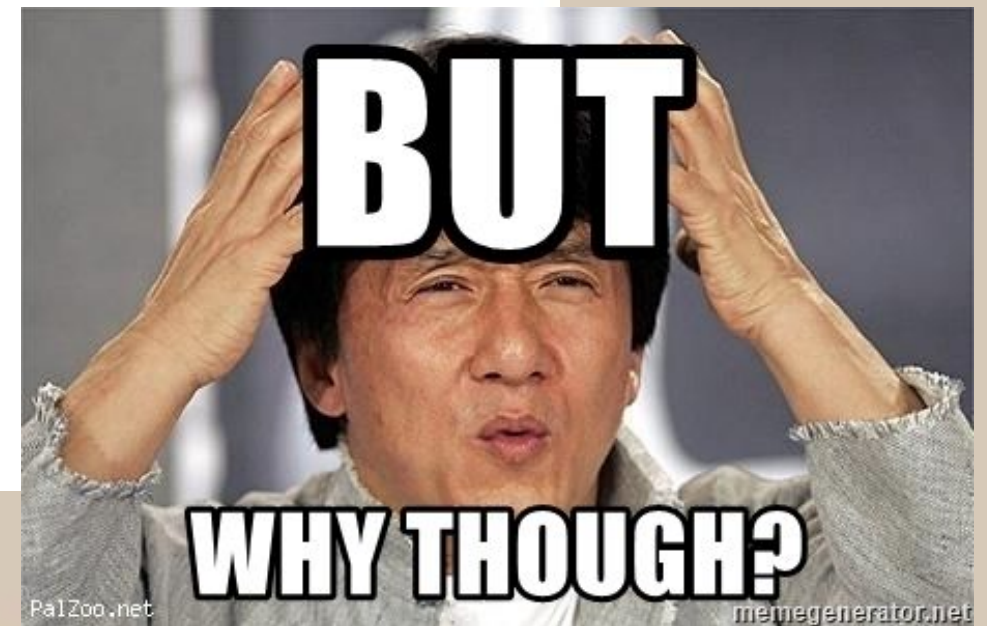
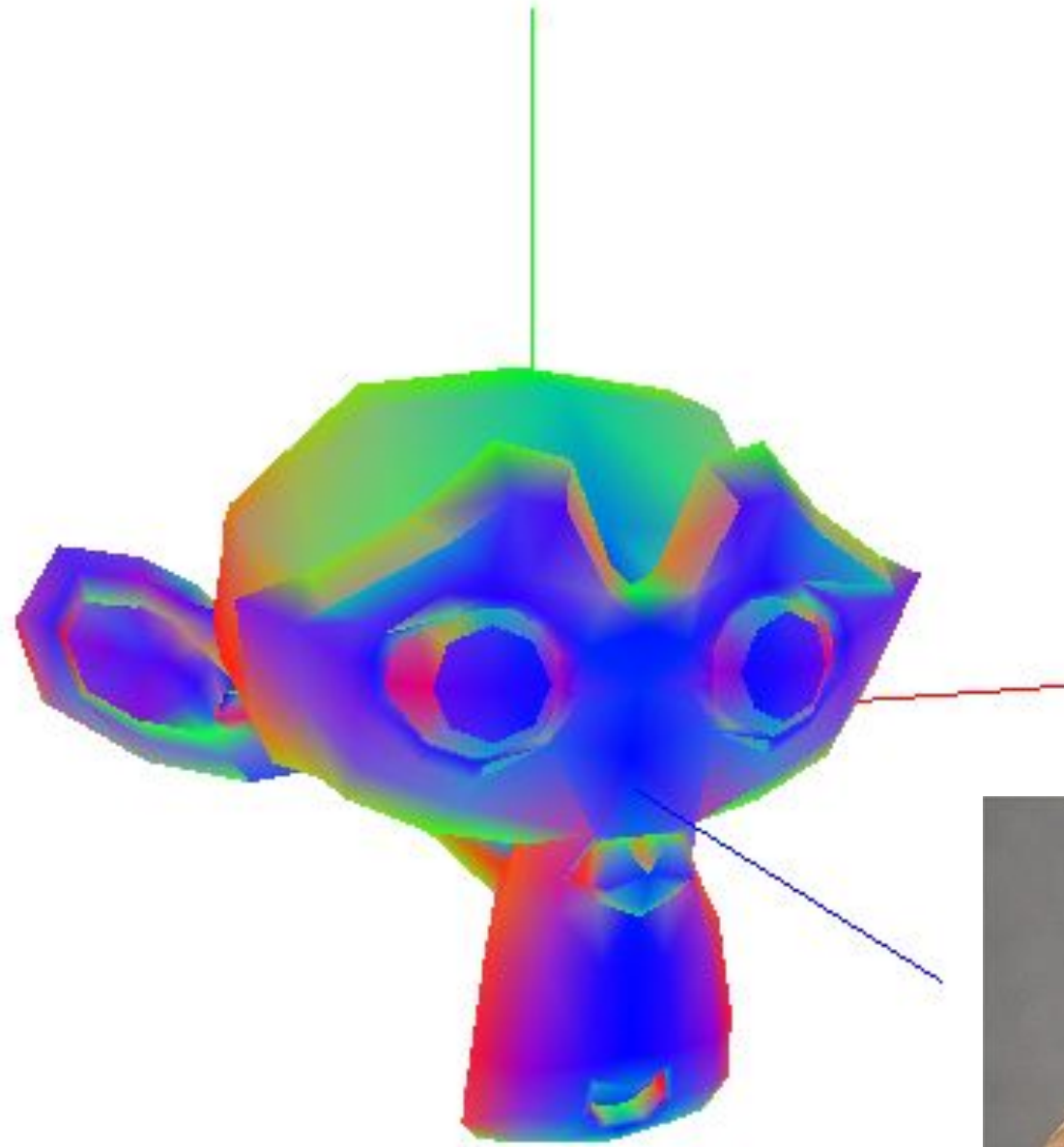
What you order...



Last of Us 2

CAPTURED FROM PS4 PRO

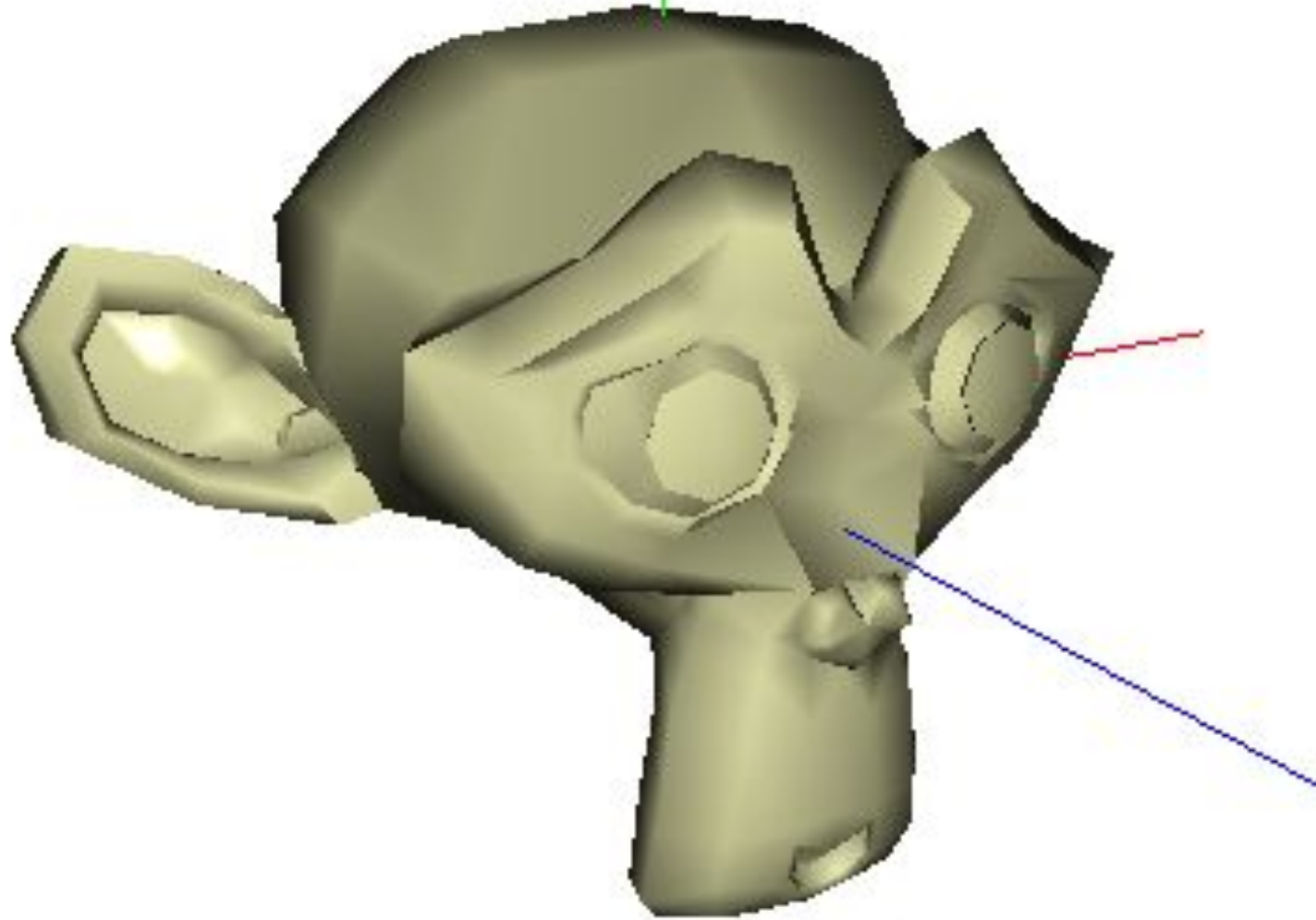
What you get...



You need to add

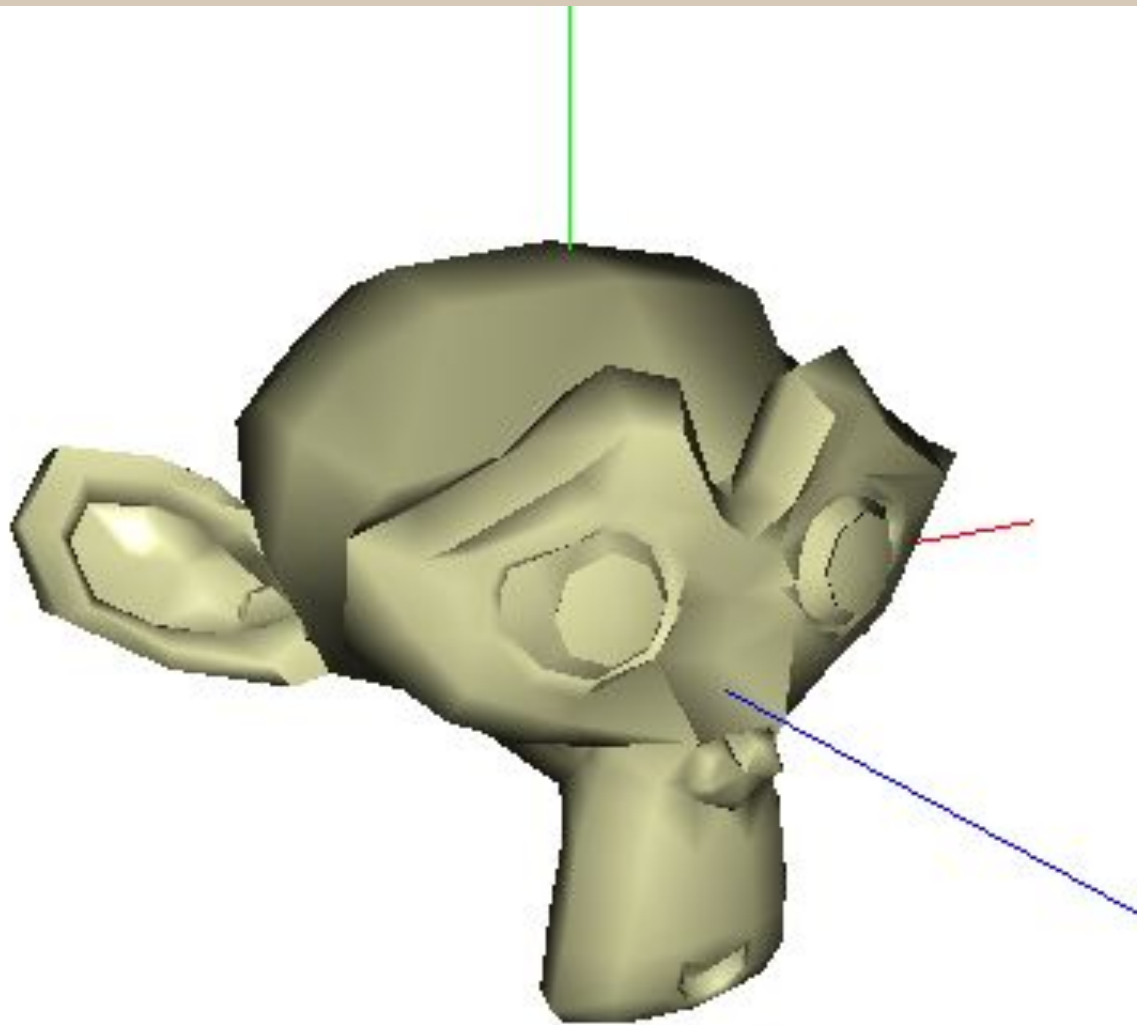
Light!!!

WOW!!!

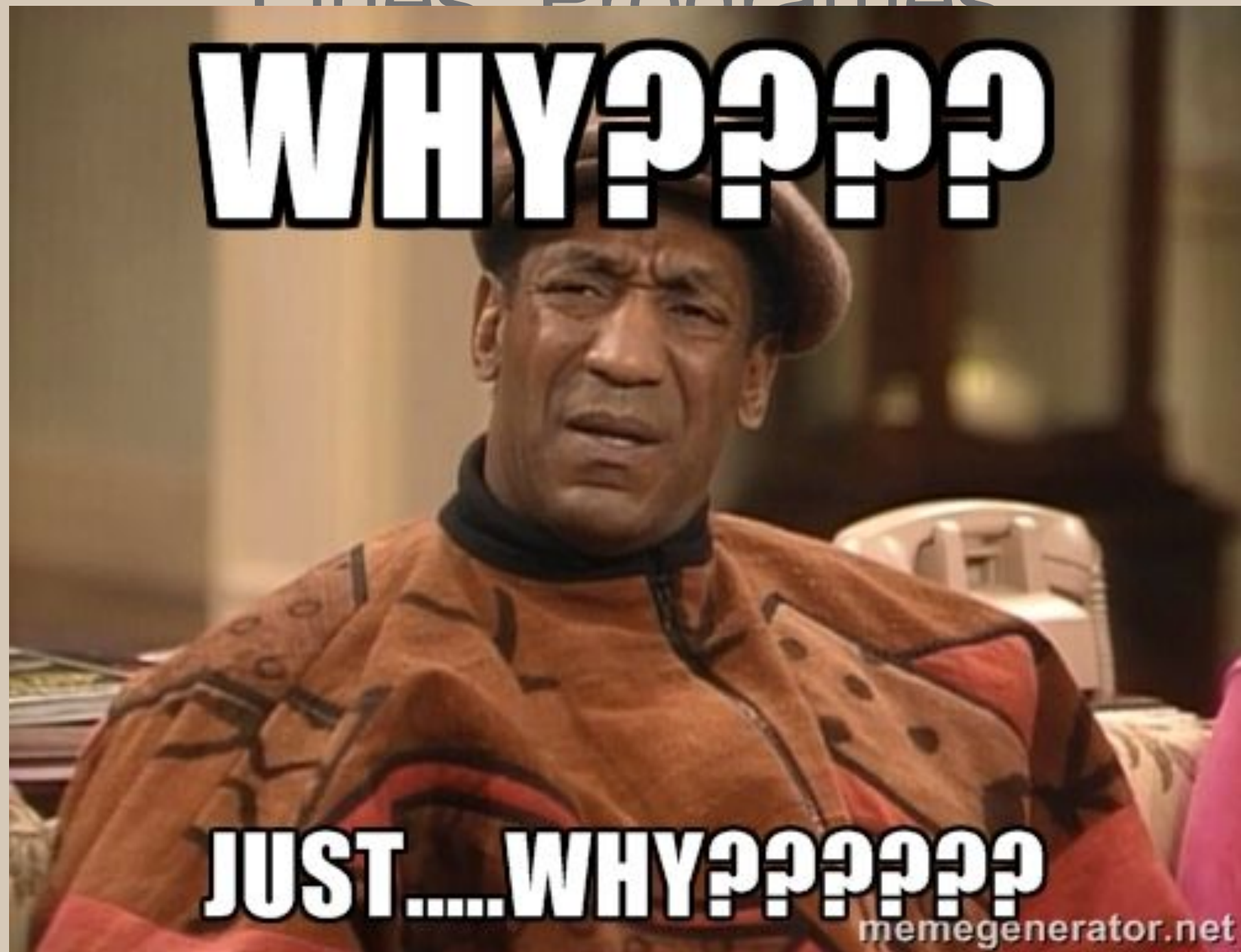


Physically-Based Rendering

Still far away from goal...



Fines Programes

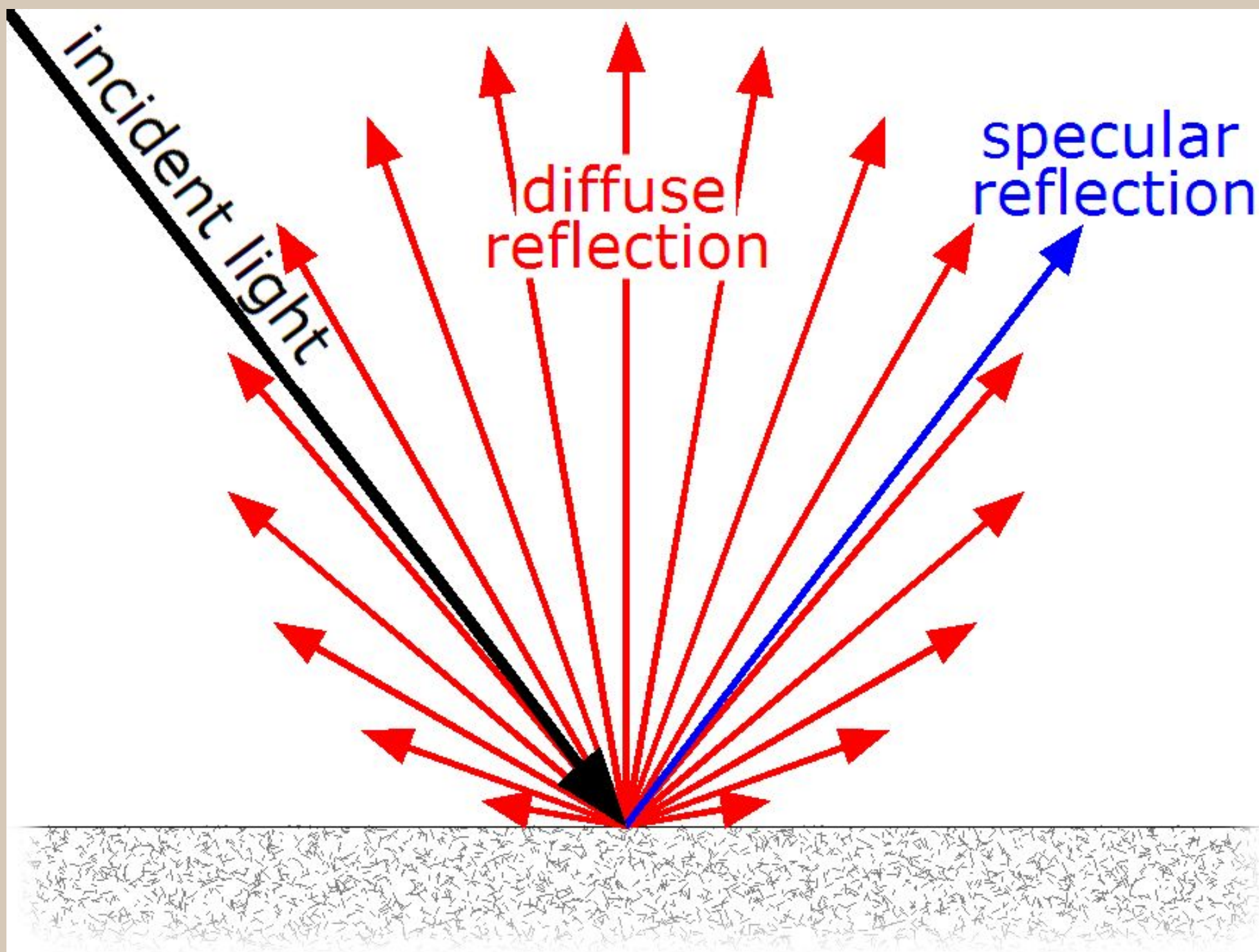


Let's talk about

Light Physics

Physically-Based Rendering

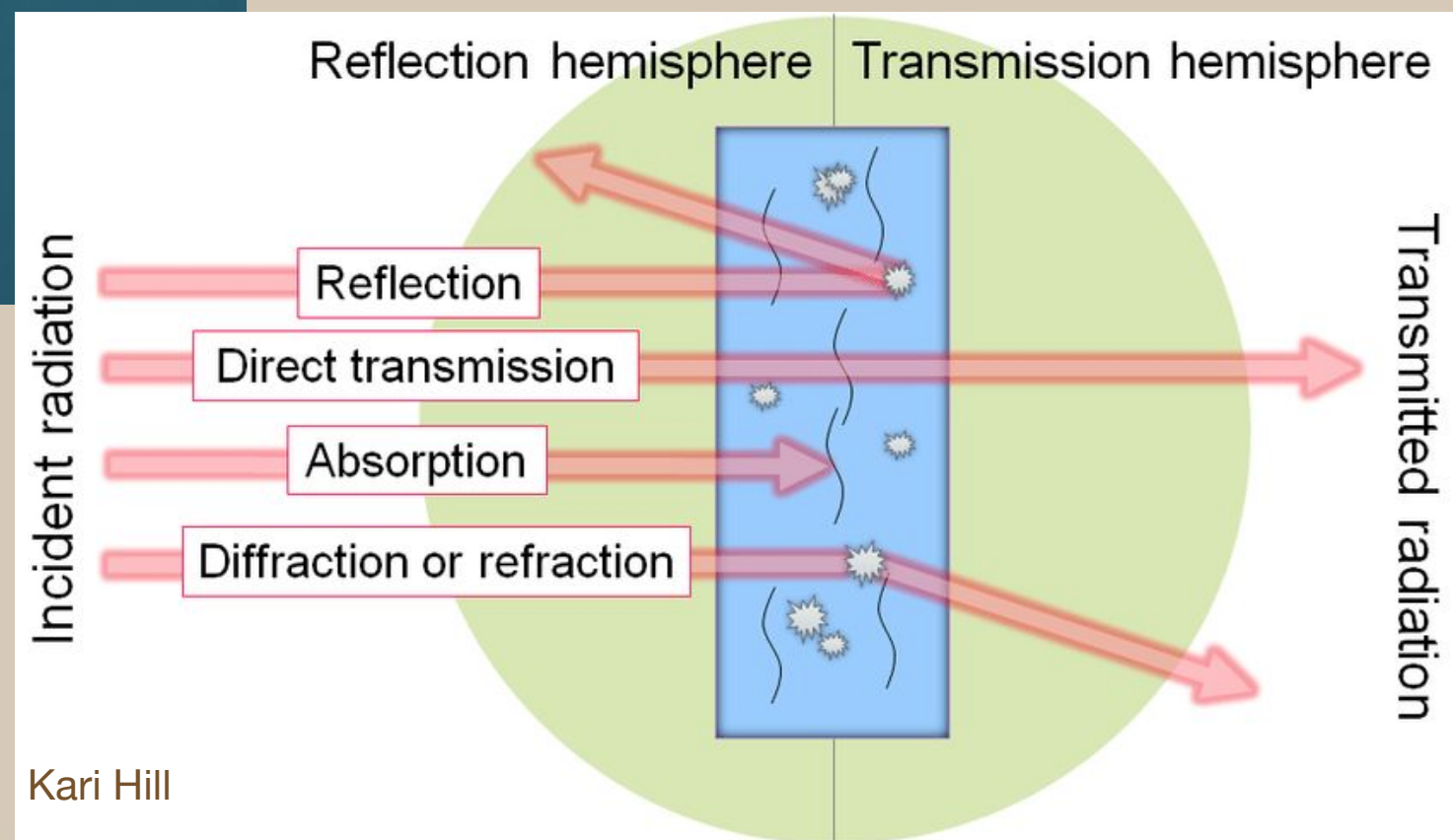
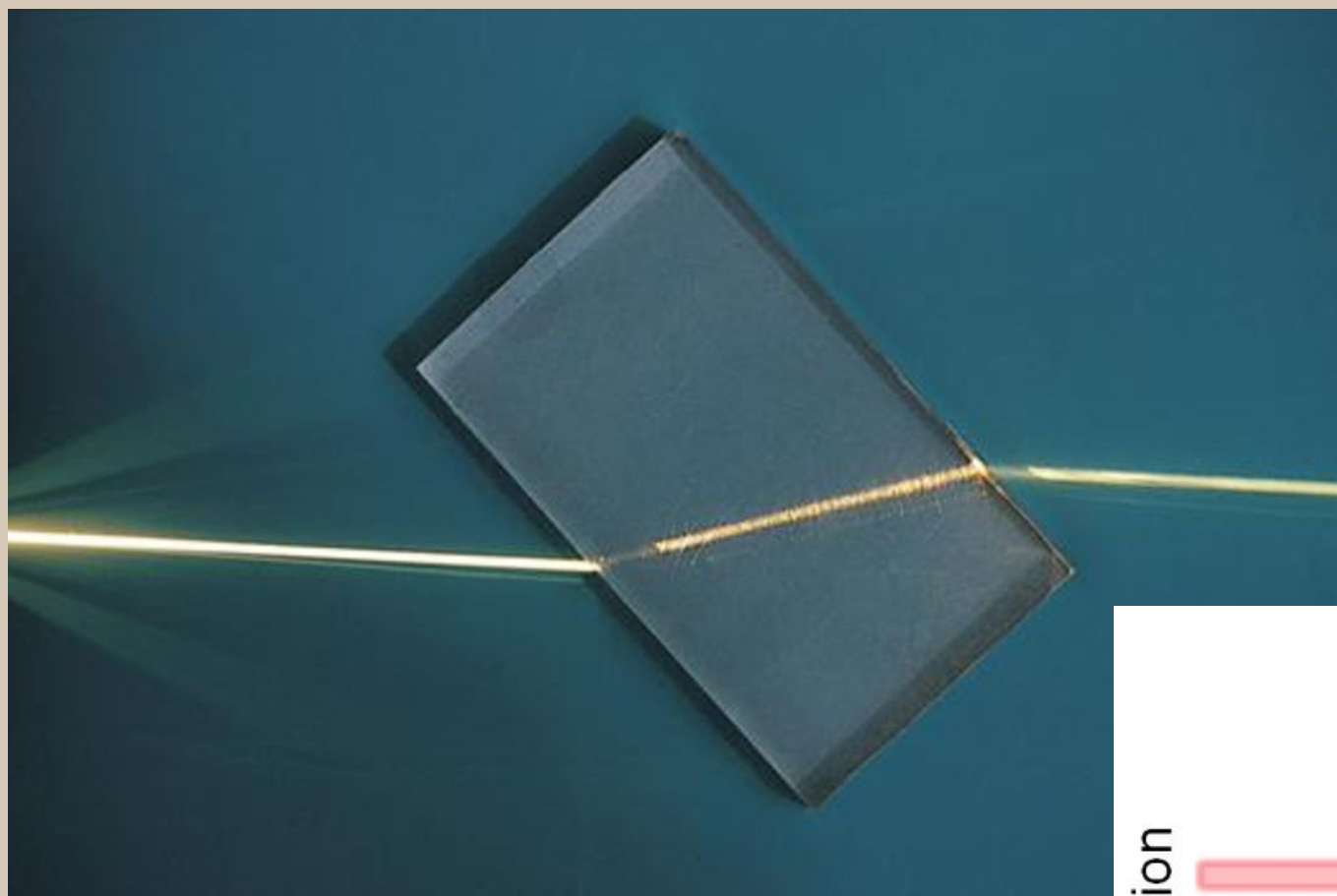
Base case



But... What about transparent surfaces?

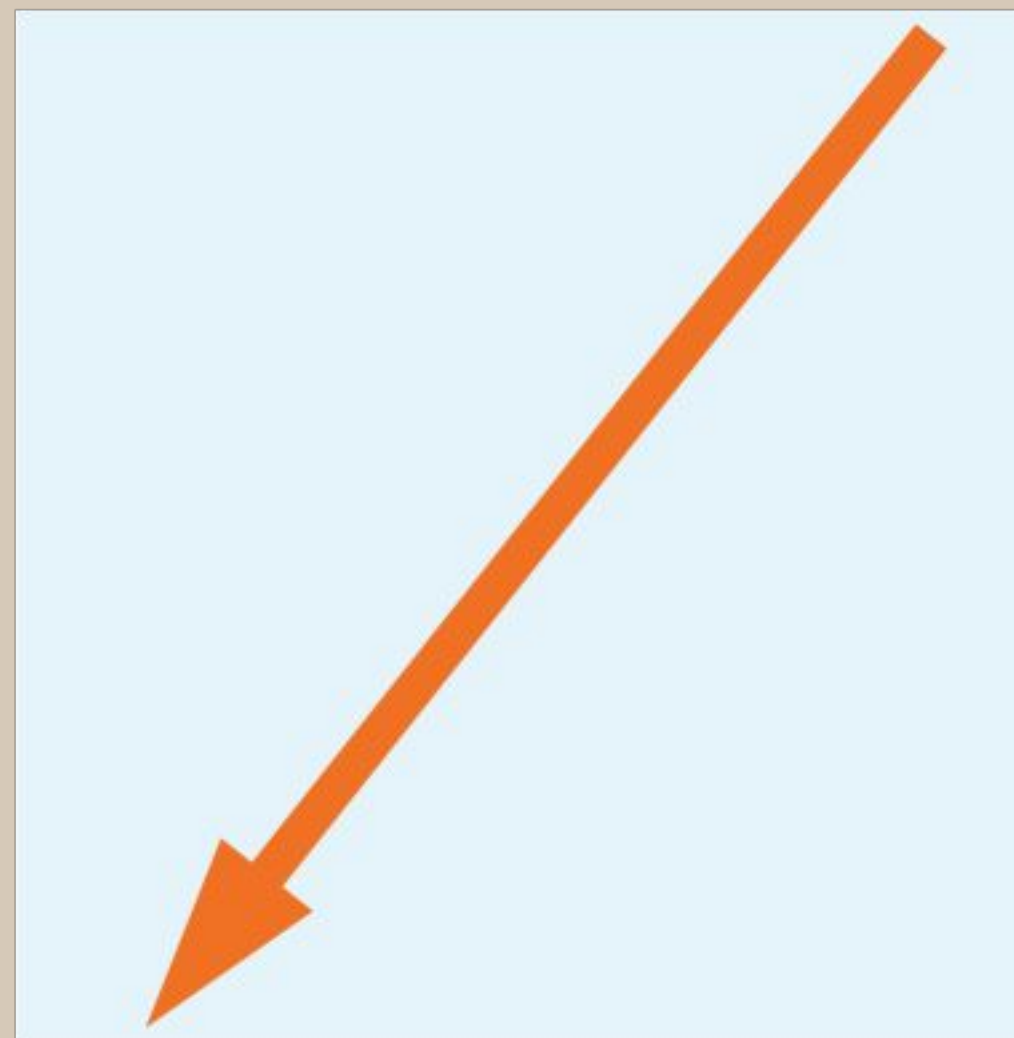


But... What about transparent surfaces?

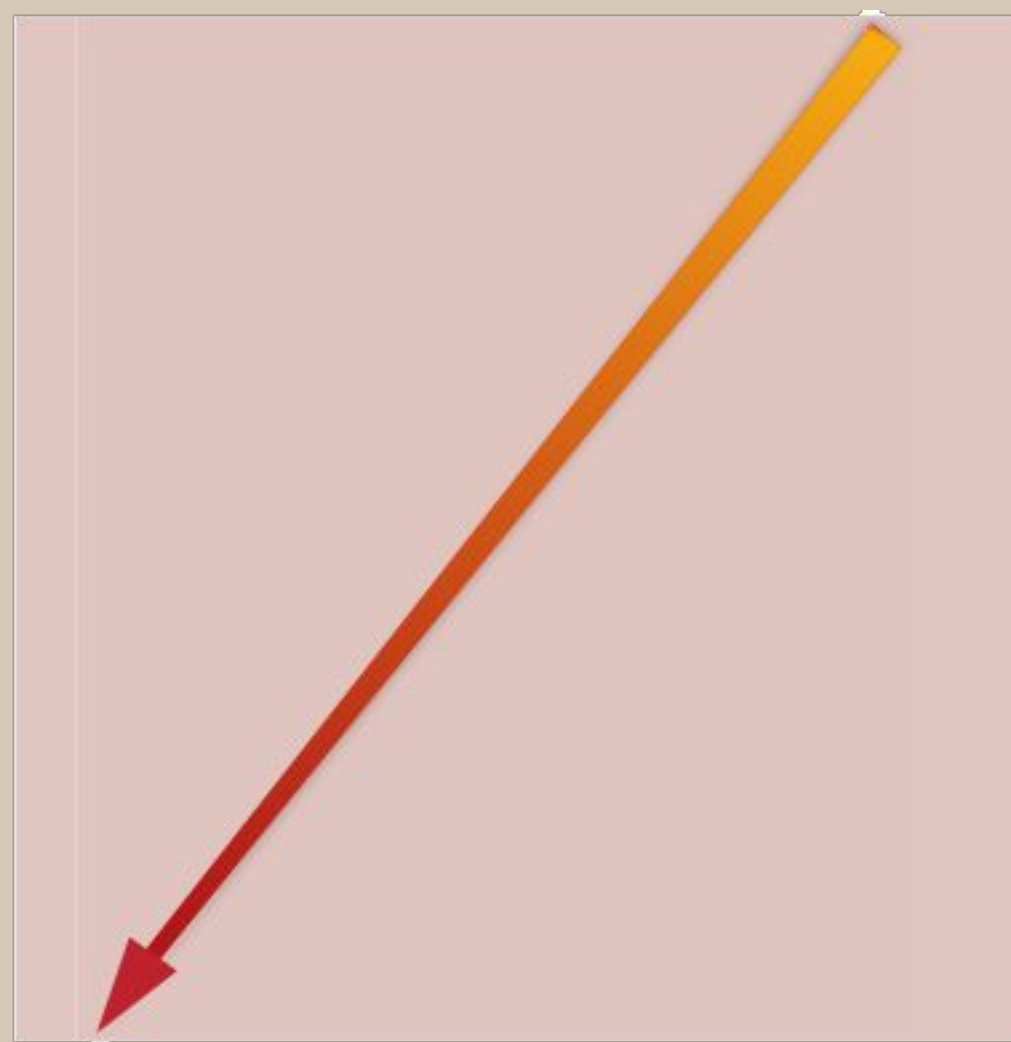


But...
does light can be affected by media?

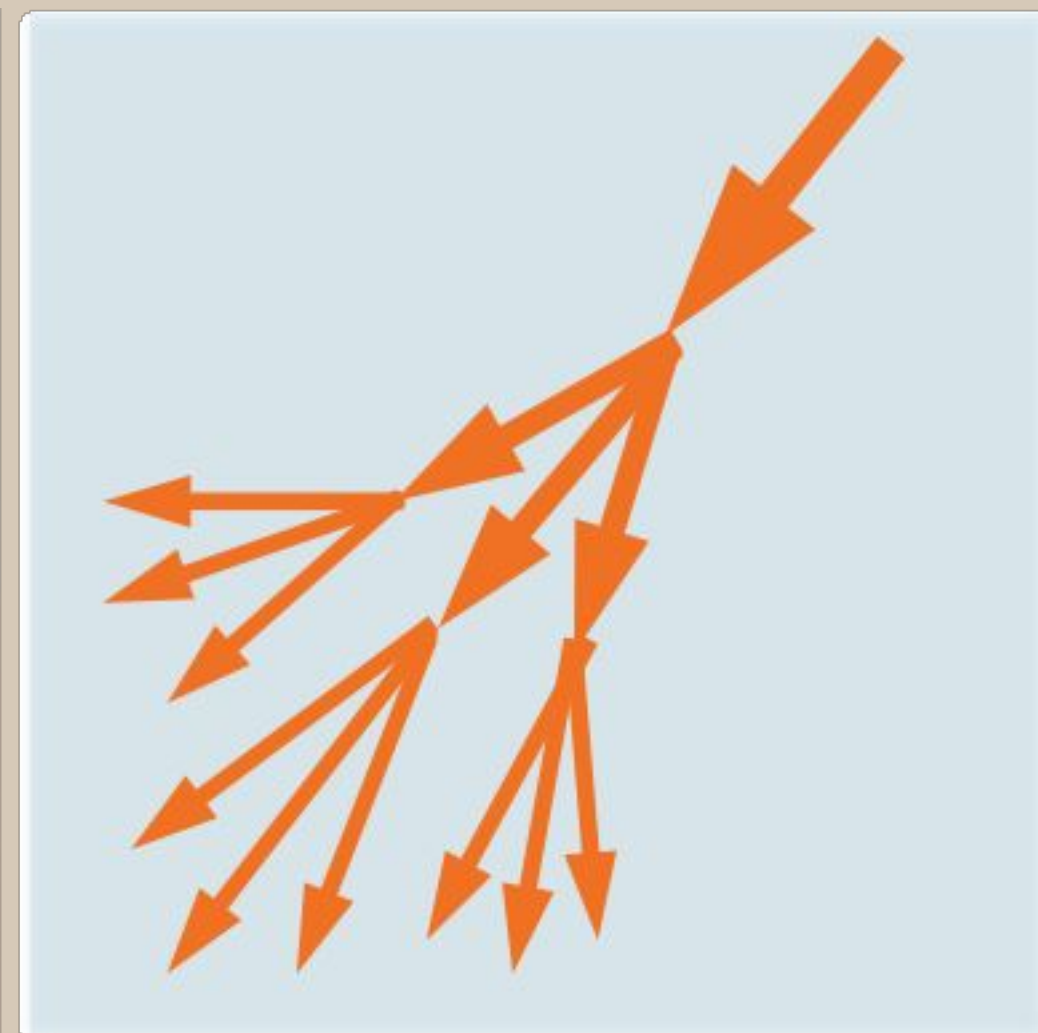
Transparent Media



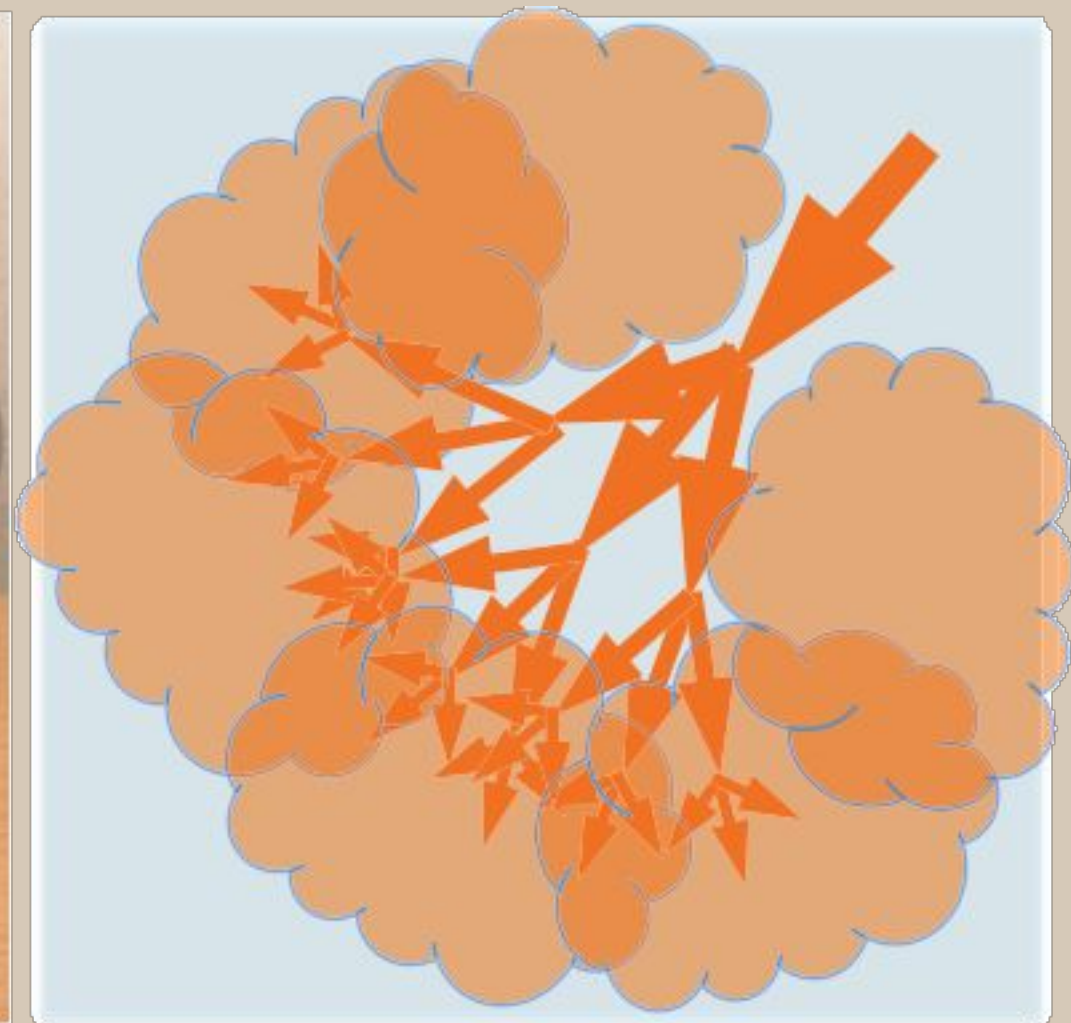
Absortion



Scattering - cloudy



Scattering - opaque



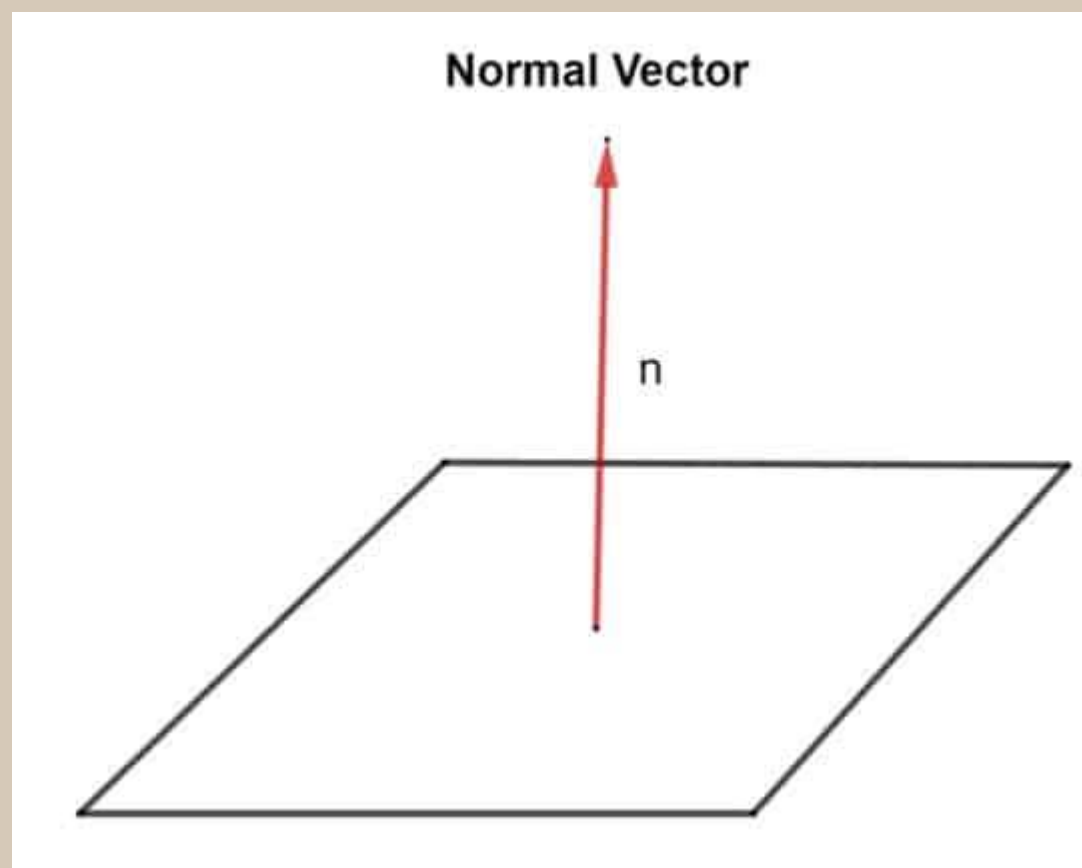
Media Interaction



Ok...

How does it work in the simplest case?

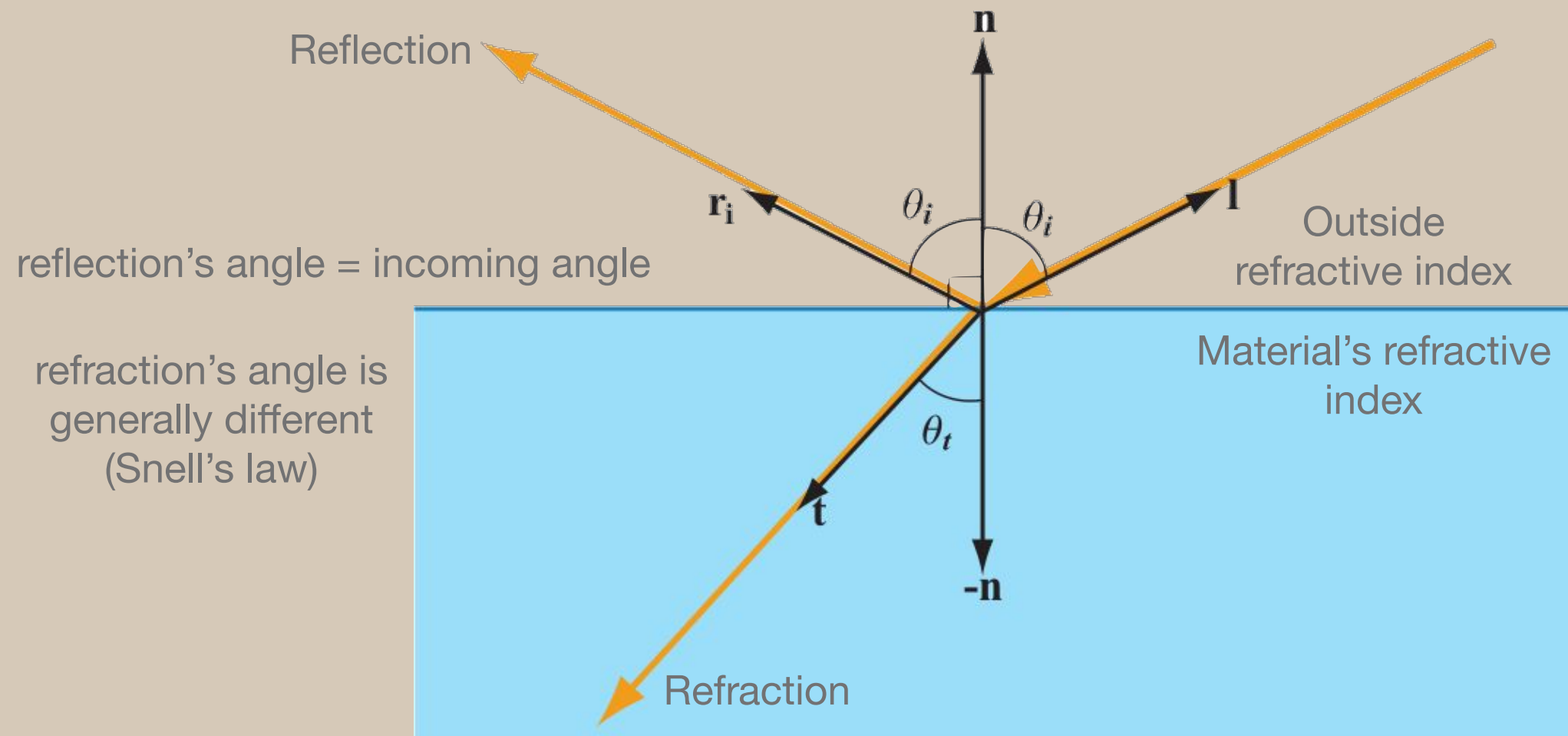
A plane



We will focus only on how light is bounced back and absorbed...

Planar Surfaces

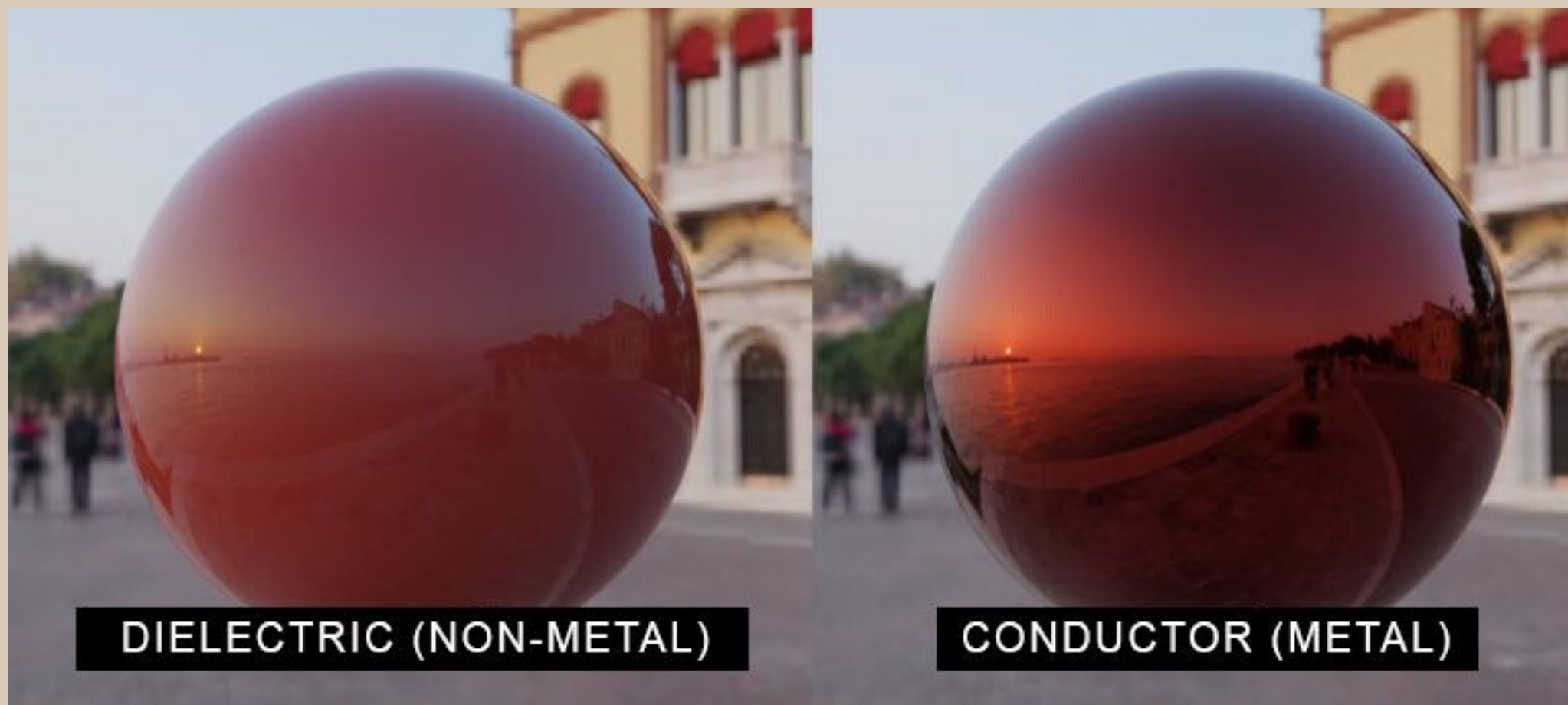
Light splits in two directions.
Fresnel tells us how much light is reflected
vs refracted



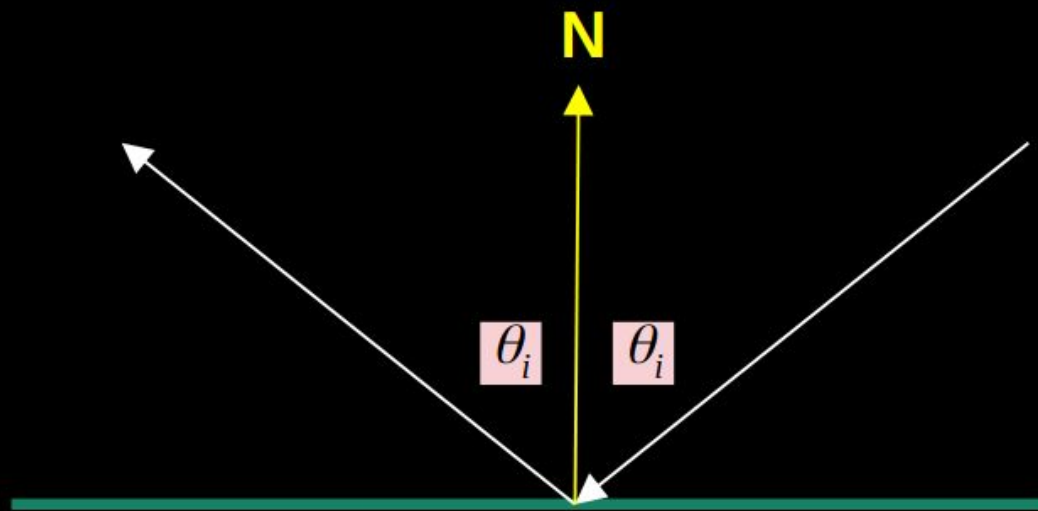
and...

what about materials?
Dielectric vs conductive

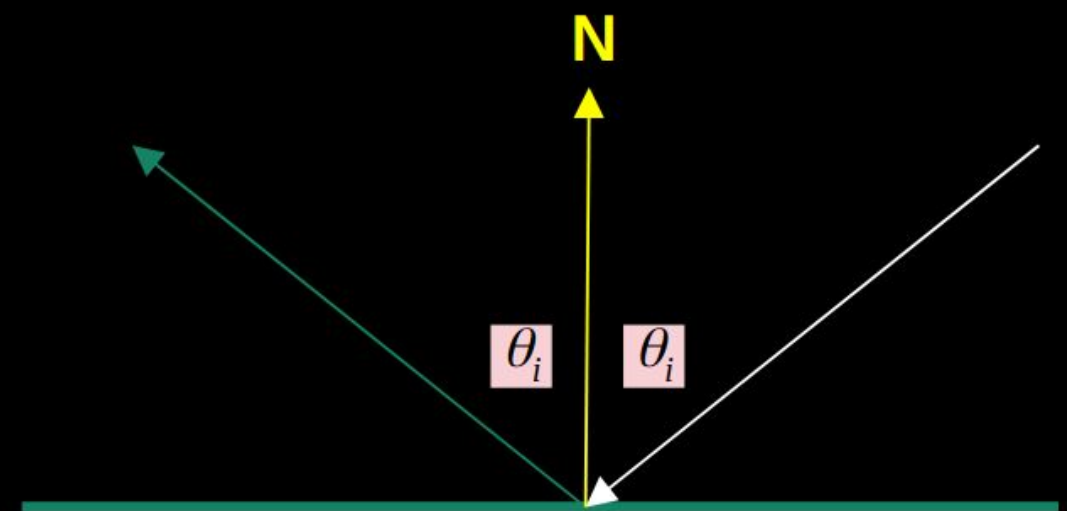
Plastic vs Metal



Reflection

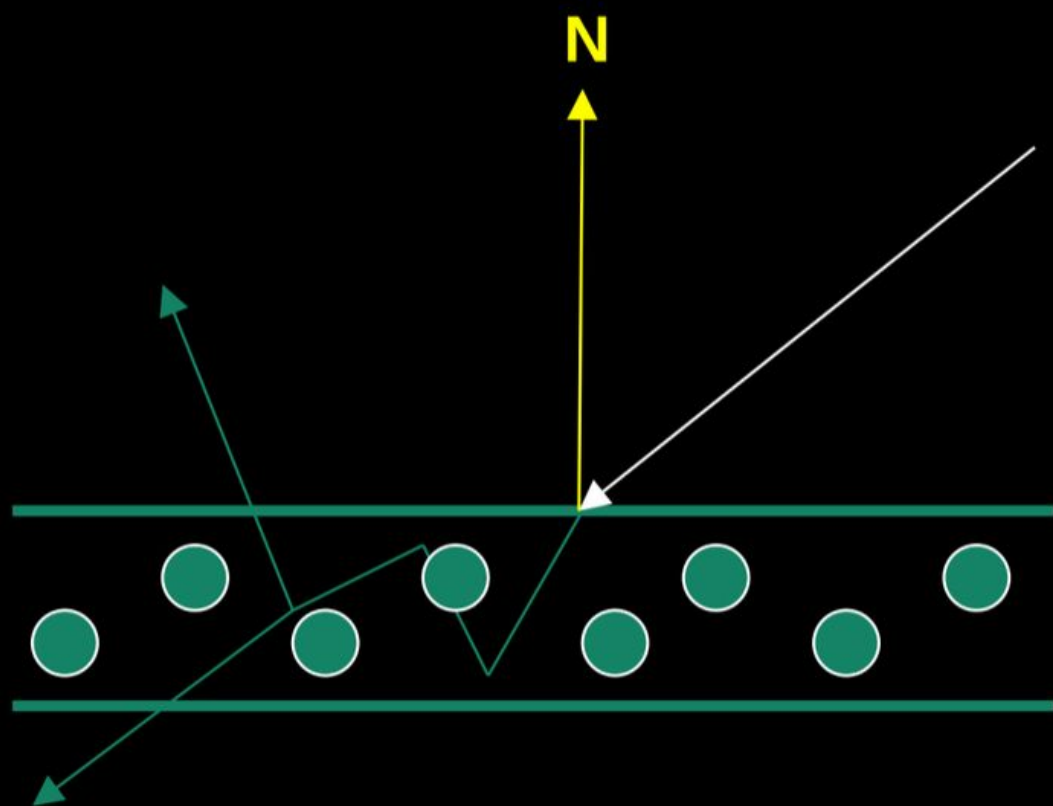


Dielectrics → reflection is NOT tinted
(light has not penetrated the surface)

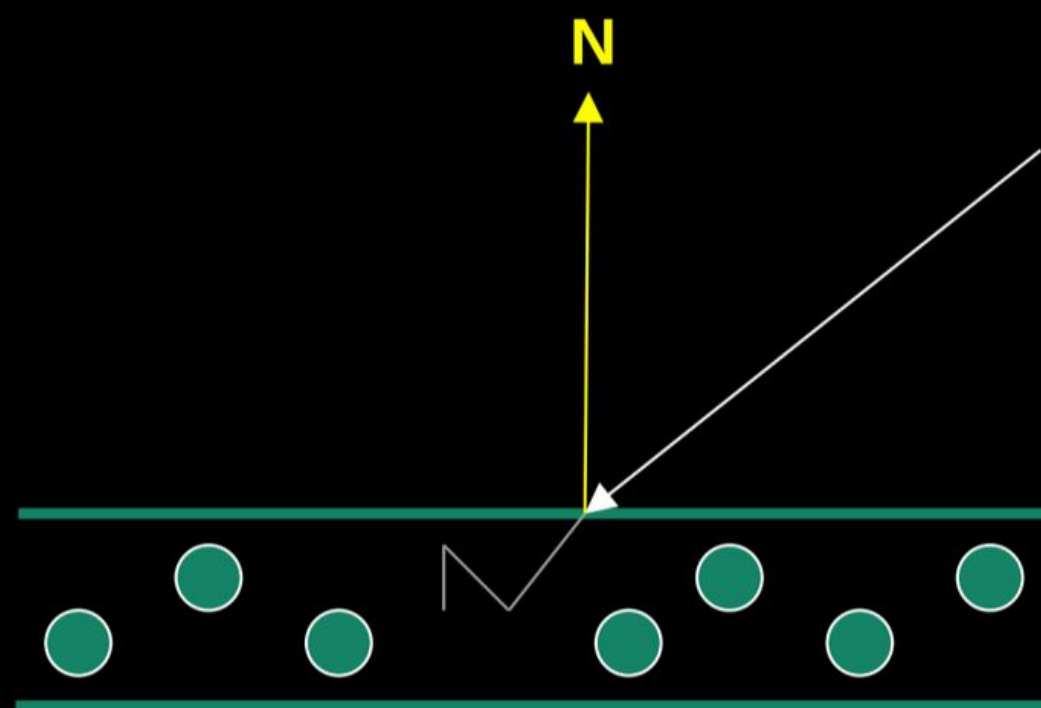


Metals (conductors) → reflection is tinted
(e.g., gold → yellow)

Refraction

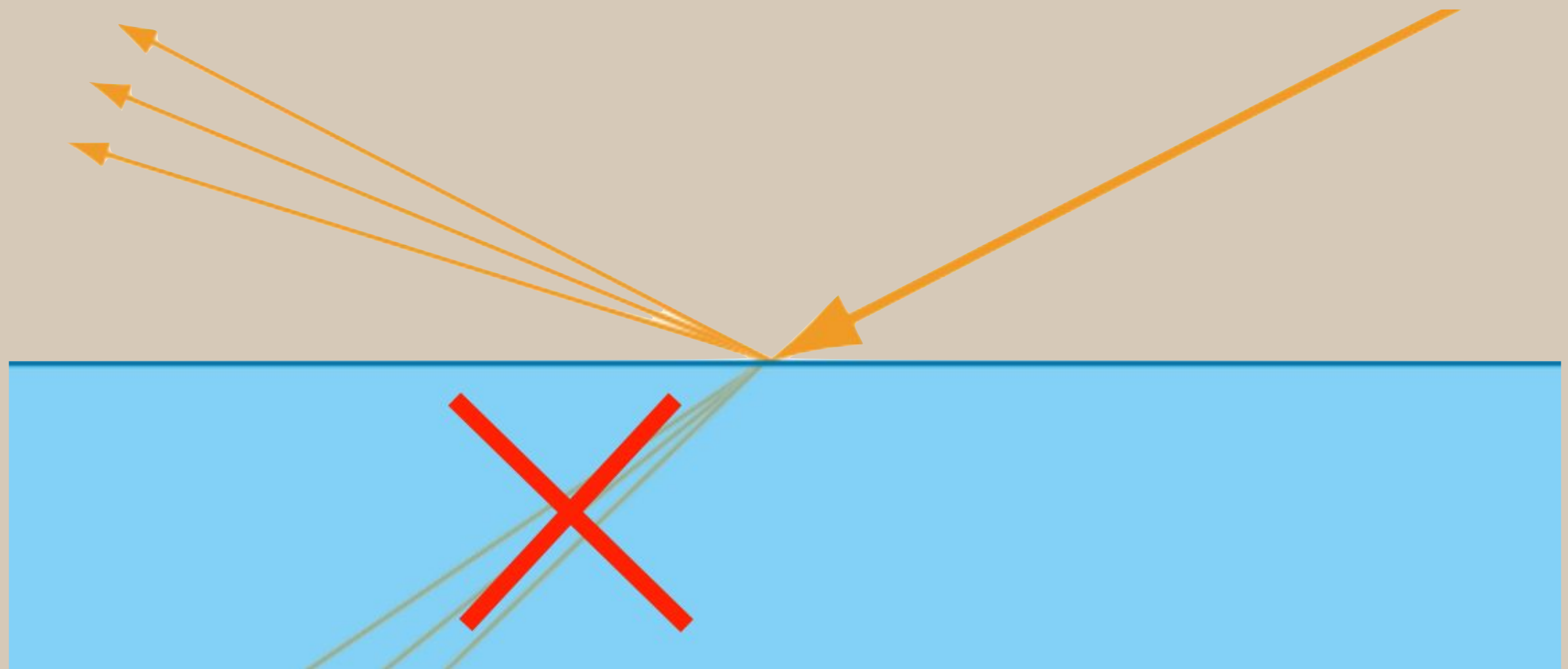


Dielectrics → refracted light is tinted, scatters back (diffuse) or forward (transmission)

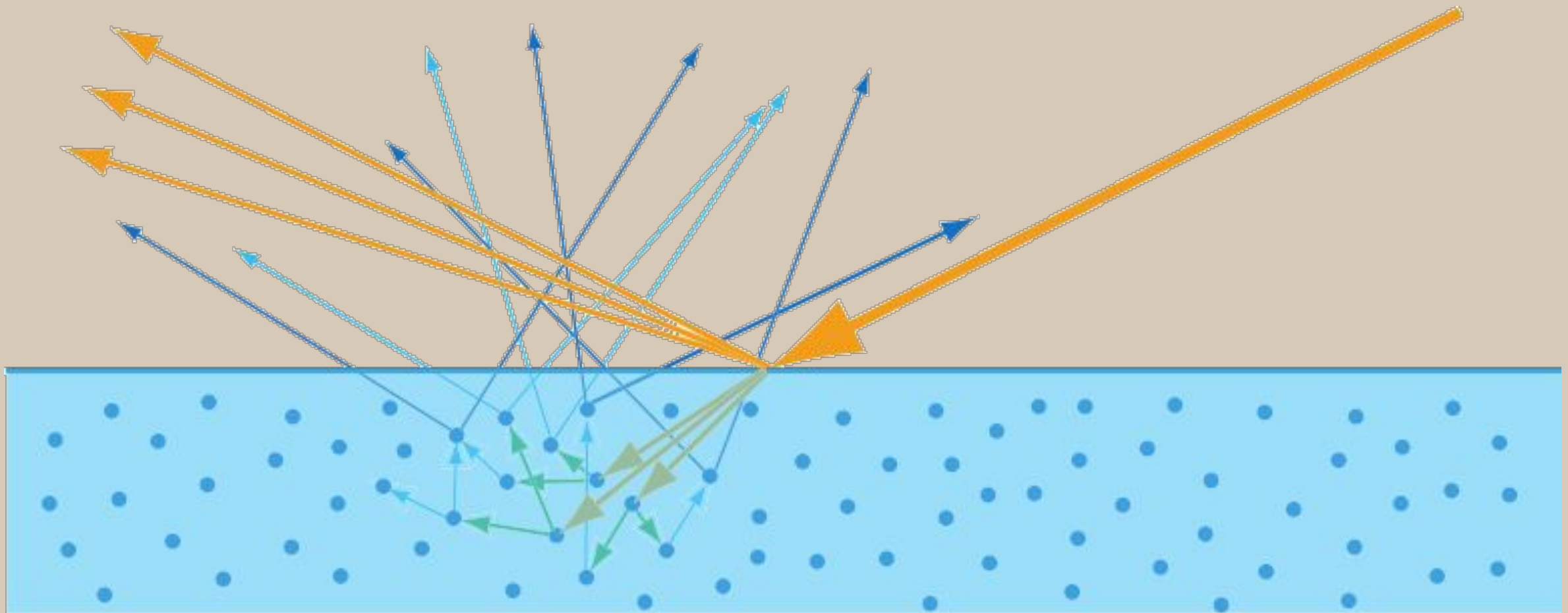


Metals (conductors) → refracted light is absorbed (by free electrons)

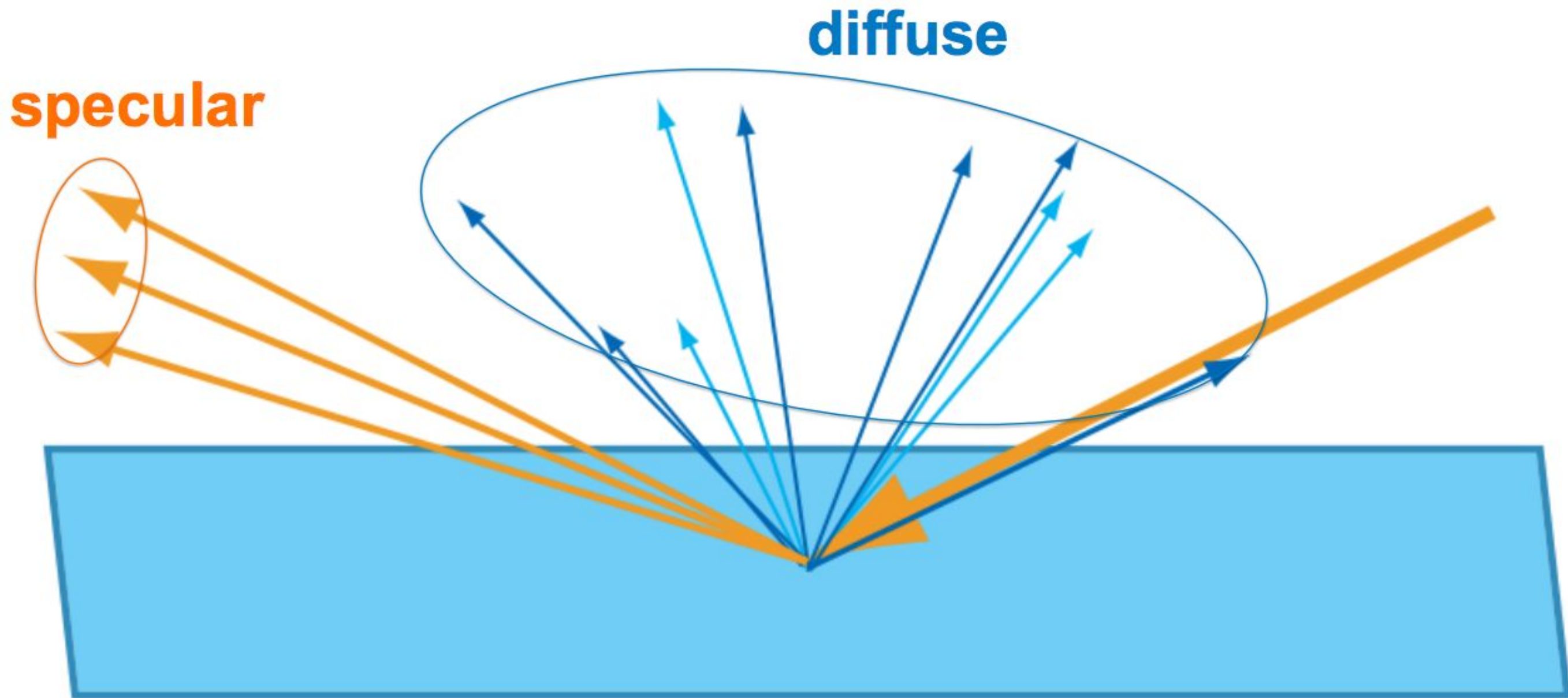
Metals



Non Metals



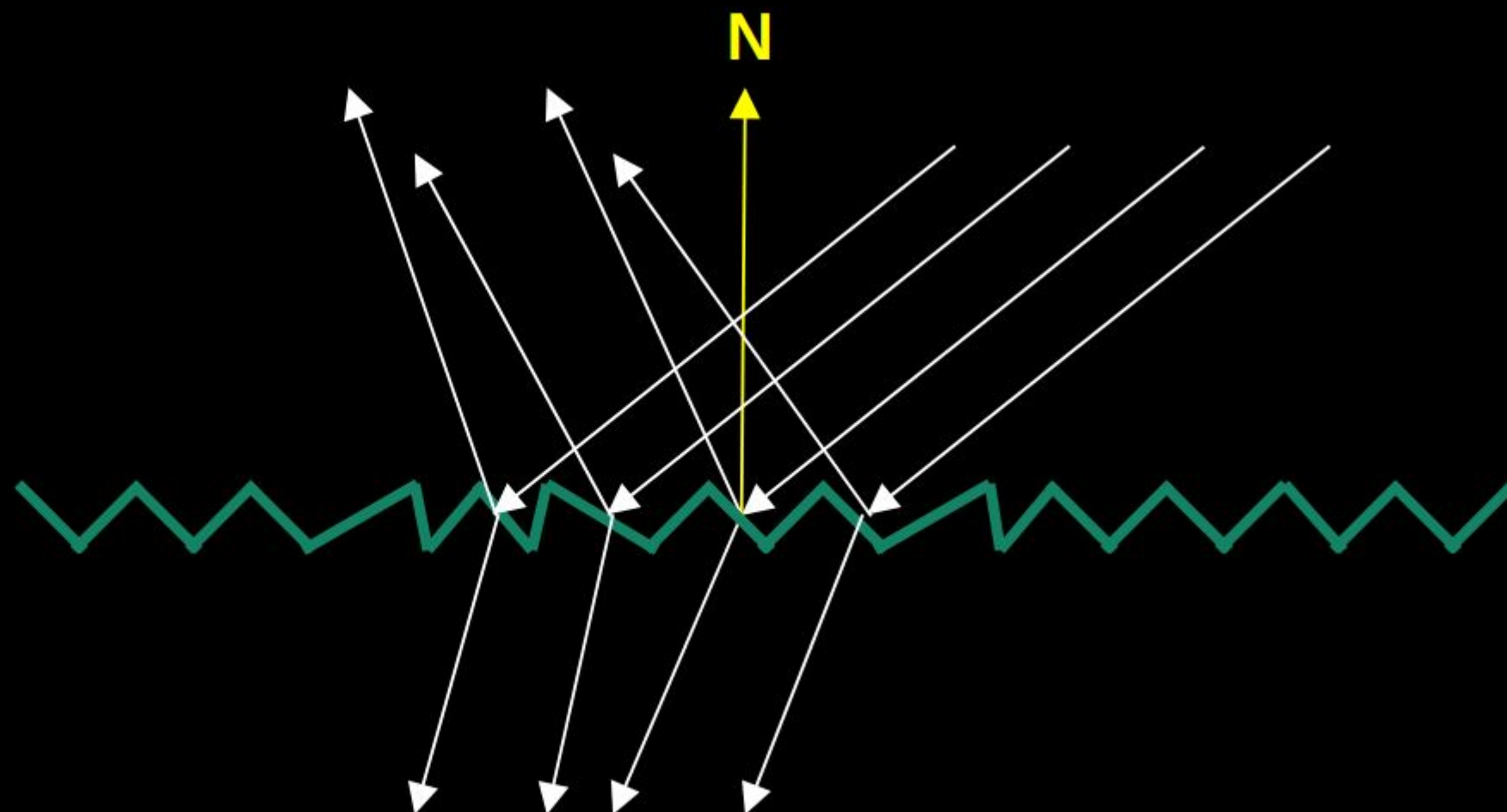
If we focus on a single point...



But...
what does it happens when surfaces
are not perfect planes?

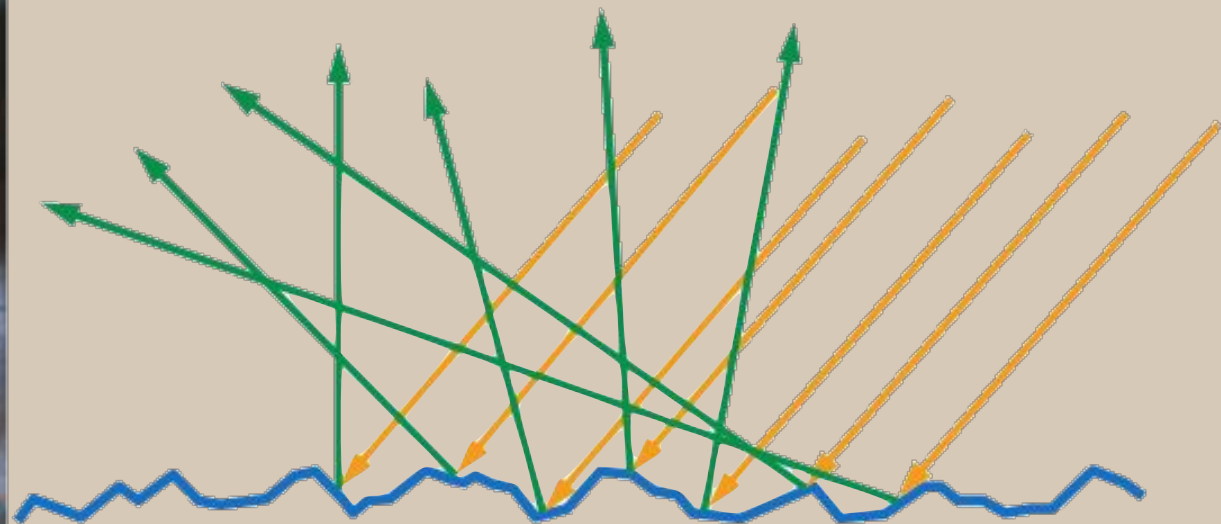
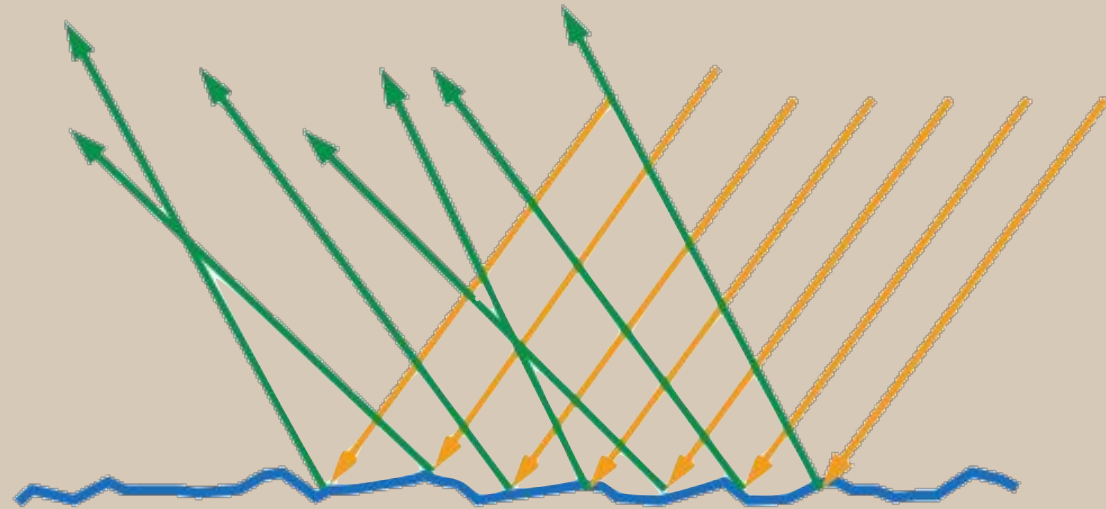
General Surfaces

We model general surfaces as a set of **microfacets**, each optically flat.



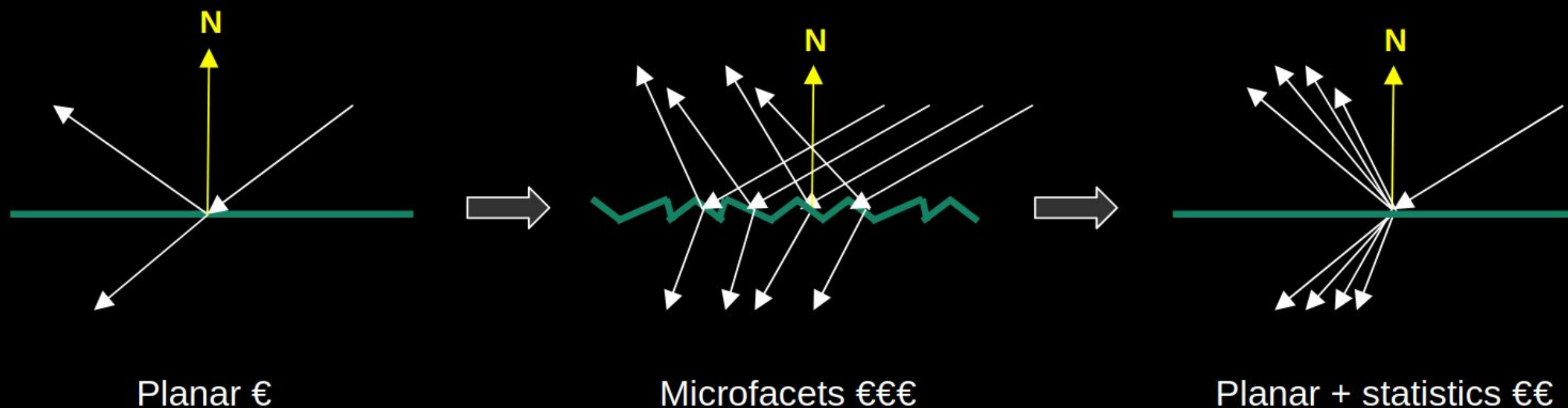
Microfacet size $<$ pixel,
but large enough to
alter appearance.

Rougher = Blurrier reflections



High Cost!

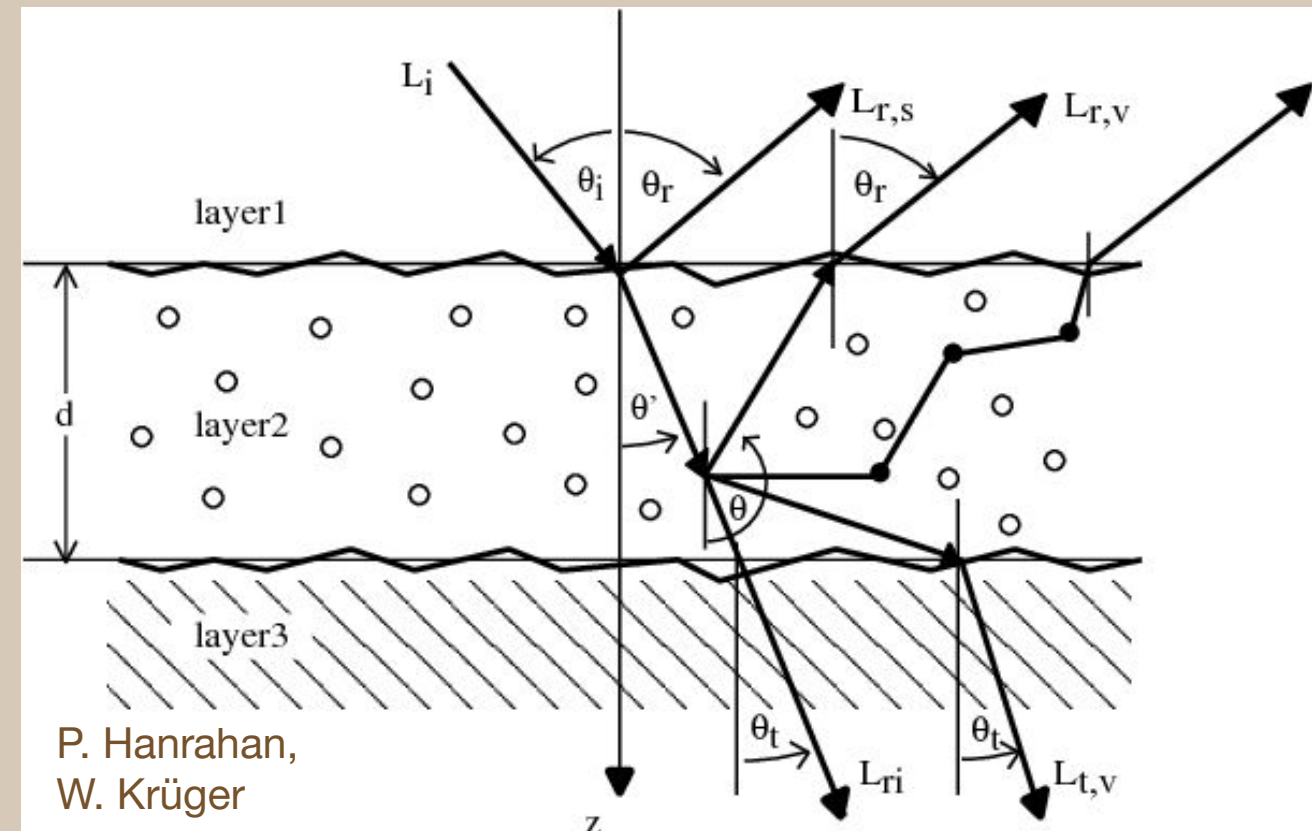
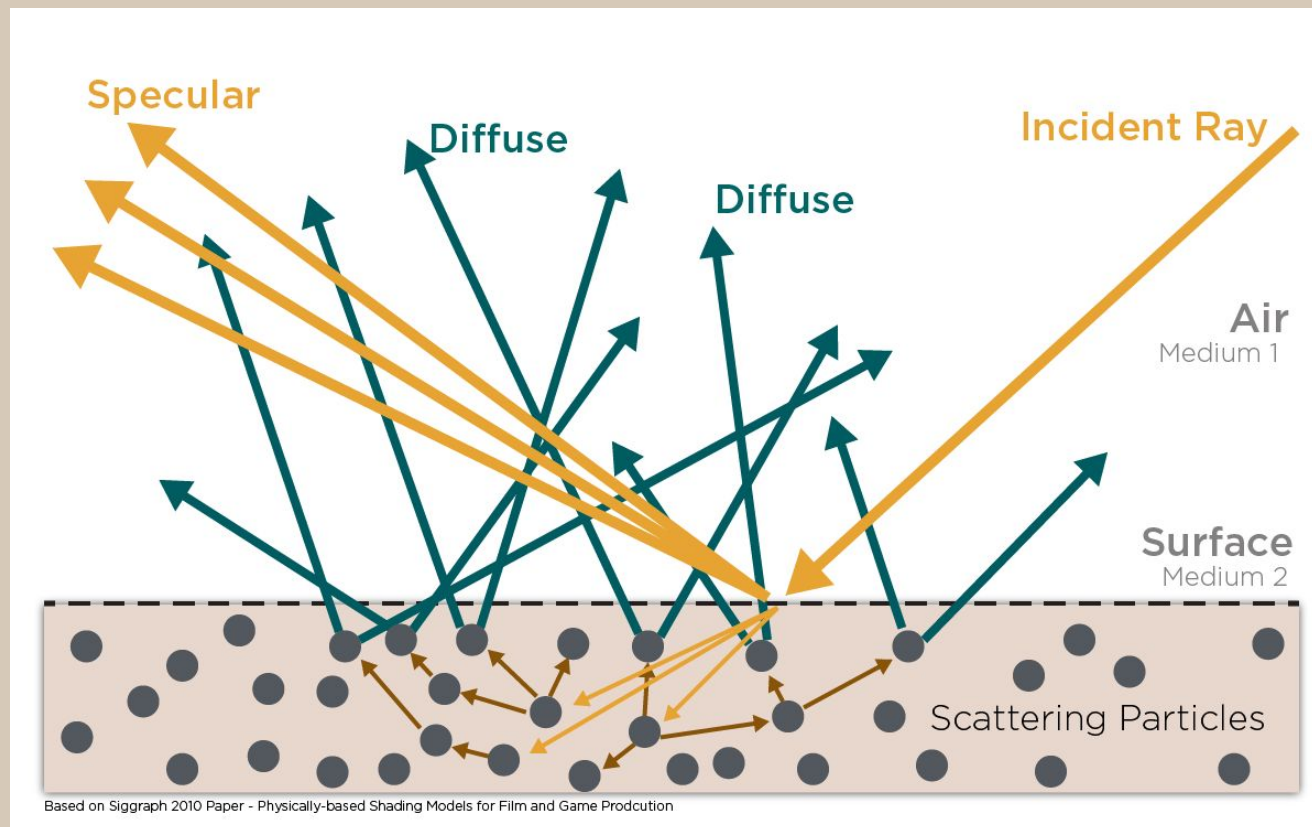
Use statistics to make computationally feasible:



Density function: describes reflecting/refracting directions.

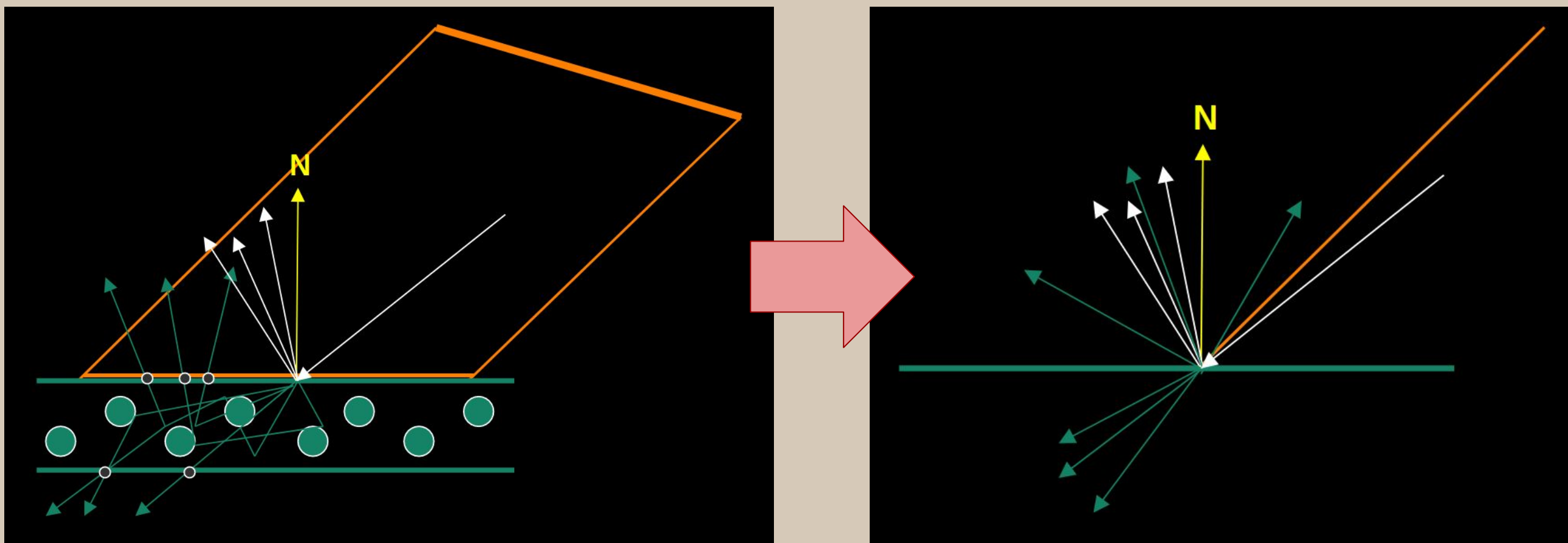
RT: sample and shoot
Shading: % contribution

Subsurface scattering



For dielectrics, refracted light bounces and forward/back-scatters through several exit points

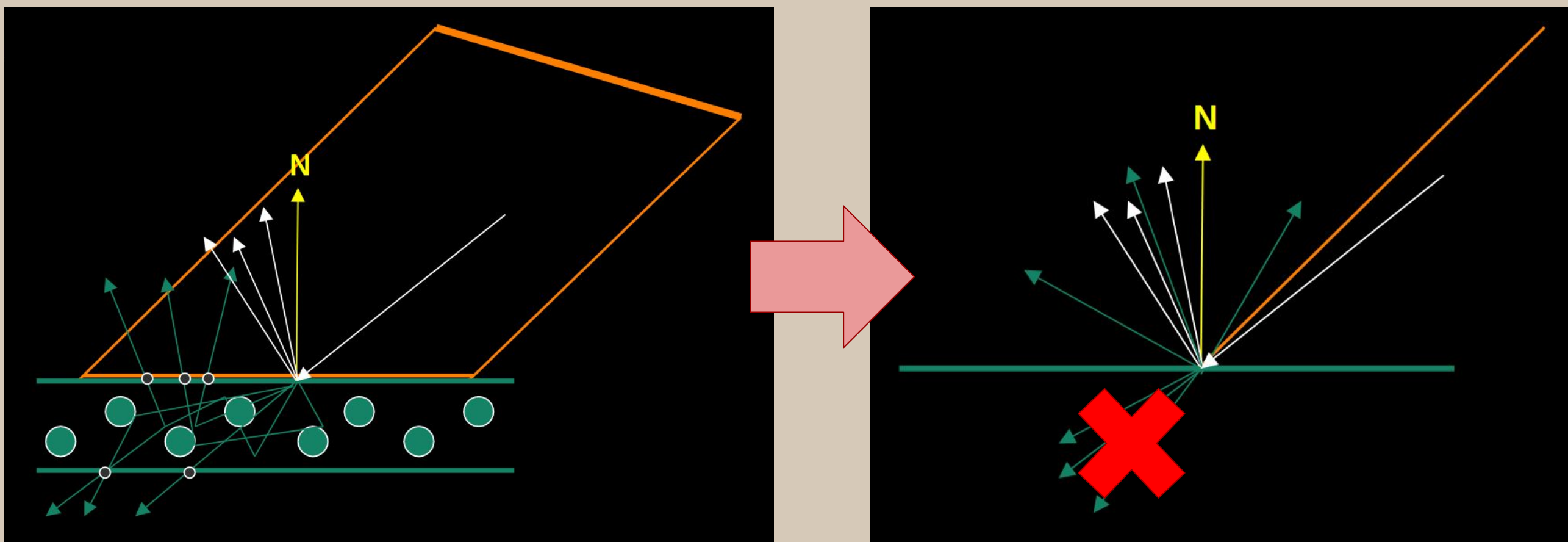
Subsurface scattering



Assume scatter area $<$ pixel size:

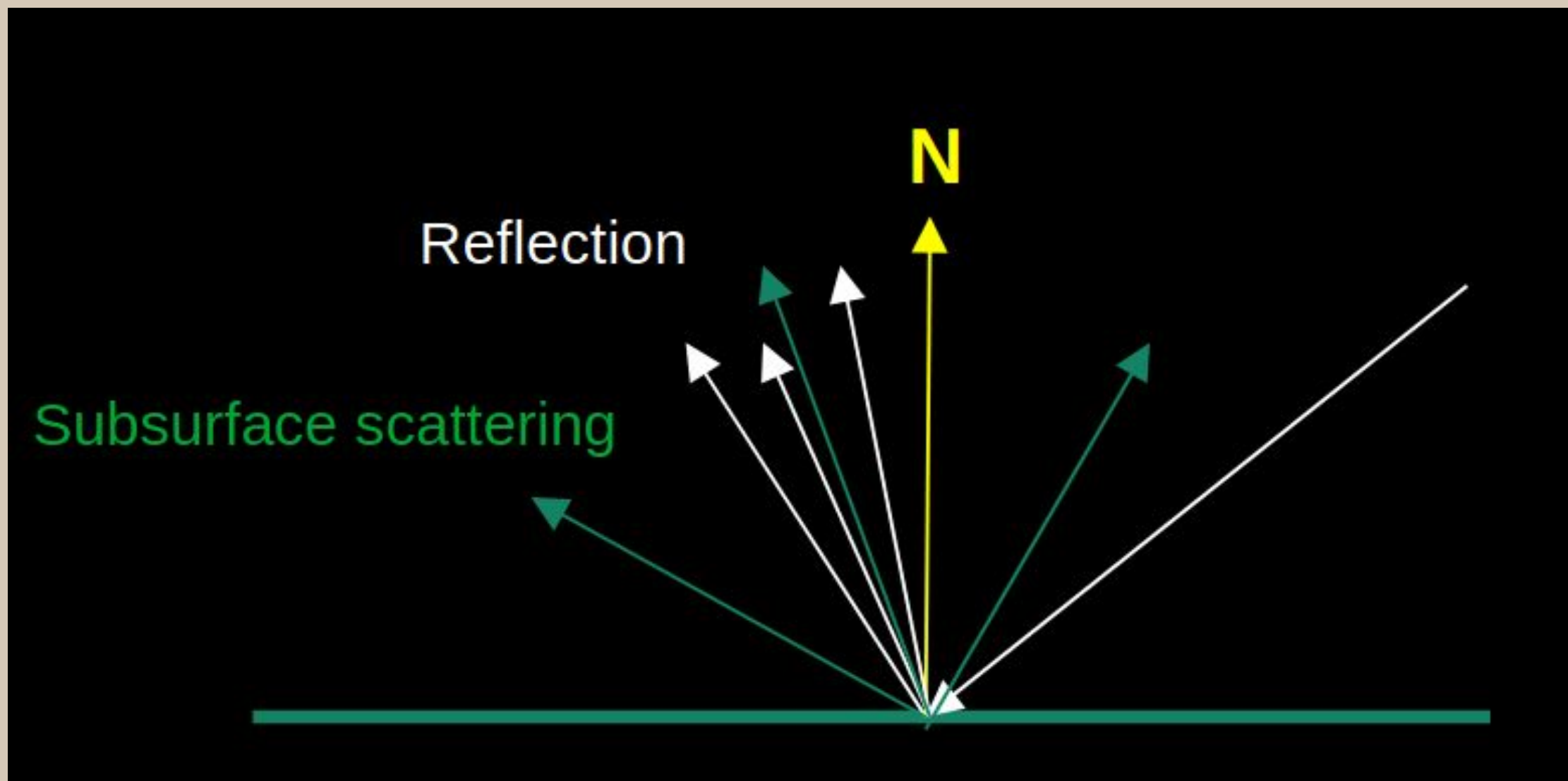
makes shading local to the point

Subsurface scattering



We assume no transmission!

Final model



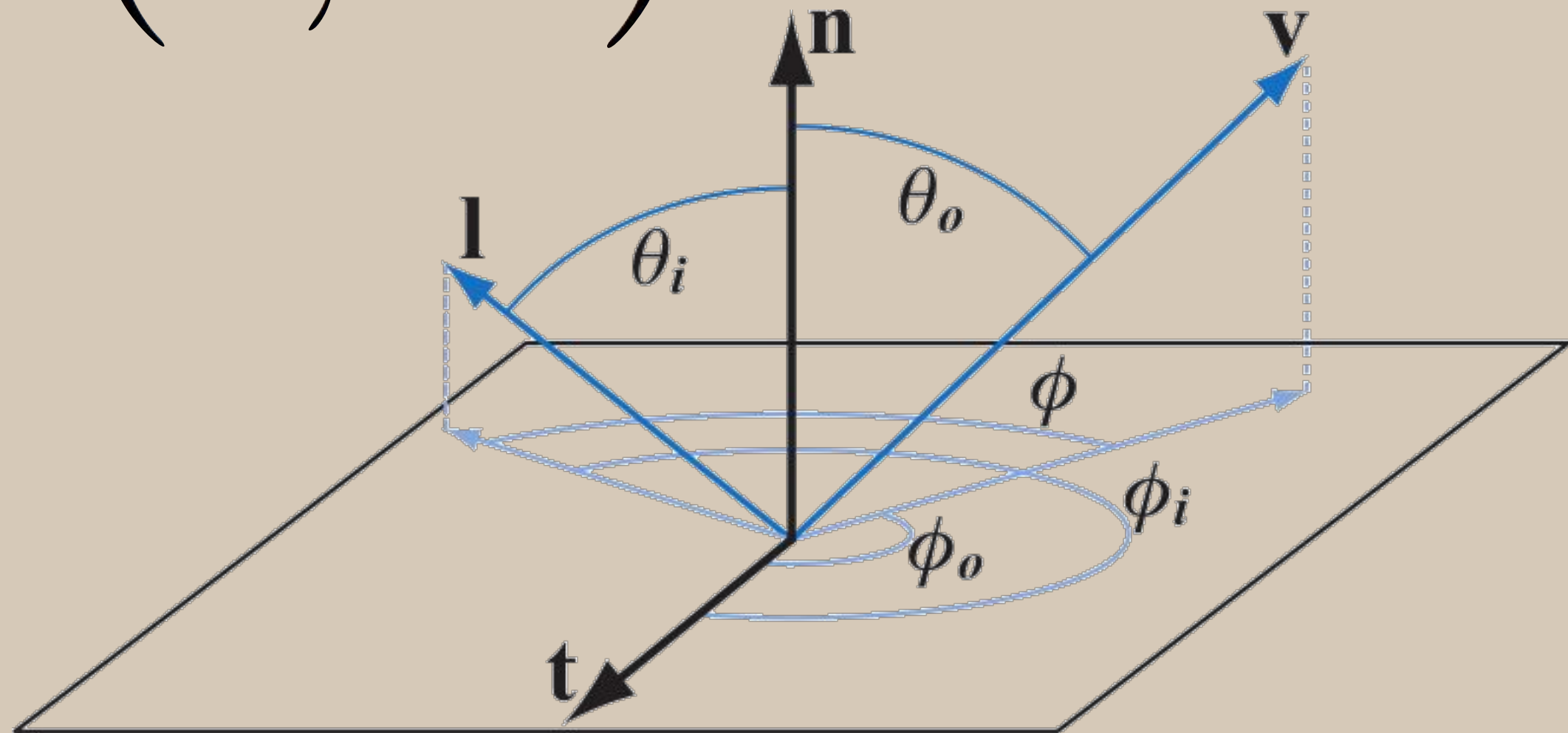
100% local
Distribution for reflection (specular)
Distribution for scattering (diffuse)

Let's talk about...

Maths!

Bidirectional Reflectance Distribution Function

$$f(\mathbf{l}, \mathbf{v})$$



Bidirectional Reflectance Distribution Function

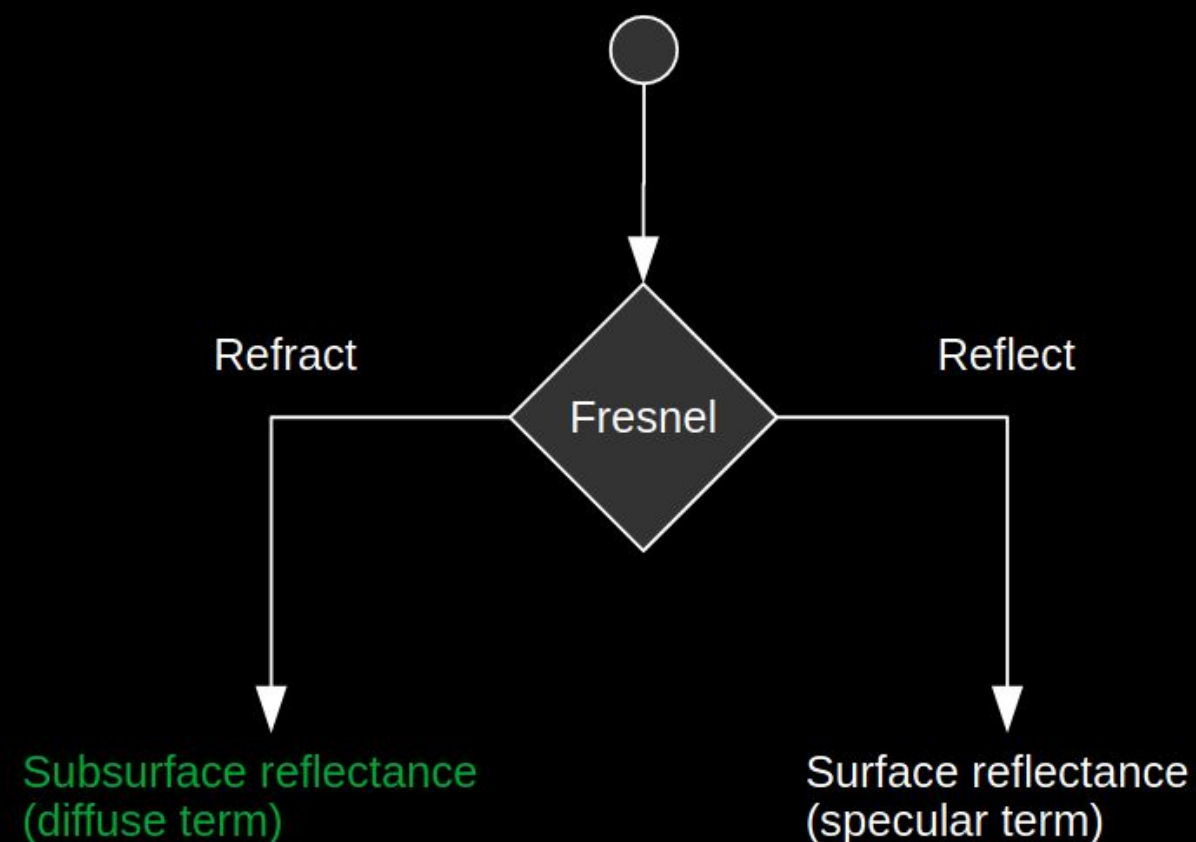
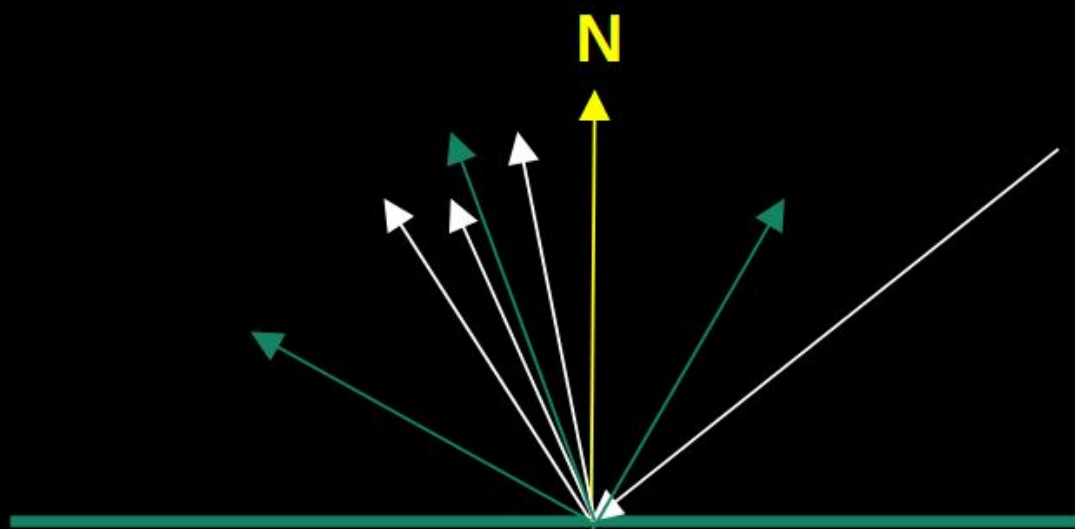
Rendering equation:

$$L_o(\mathbf{v}) = \int_{\Omega} f(\mathbf{l}, \mathbf{v}) \otimes L_i(\mathbf{l})(\mathbf{n} \cdot \mathbf{l}) d\omega_i$$

Component-wise RGB multiplication

Shading Model

BRDF = Diffuse + Specular, each modulated (shading) or conditioned (path tracing) by Fresnel.



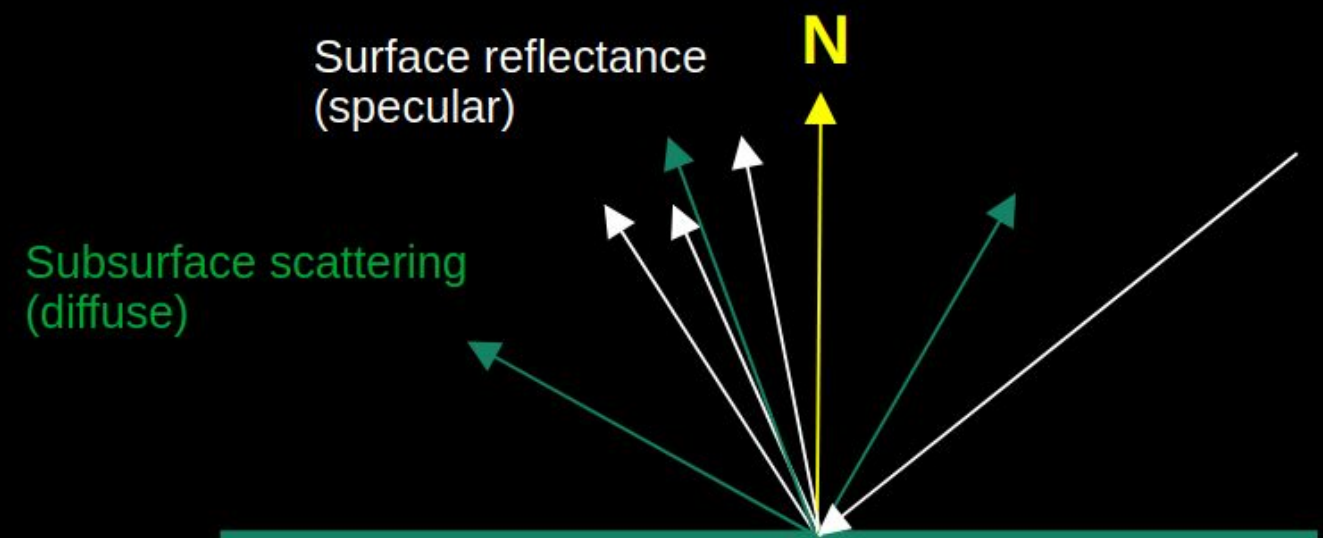
Cook-Torrance BRDF

$$f_r = k_d F_{Lambert} + k_s F_{Cook - Torrance}$$

We will look at the widely-used Cook-Torrance BRDF.

Many alternatives exist resulting from picking different diffuse and specular terms.

Quality-performance trade-off.



Diffuse Term

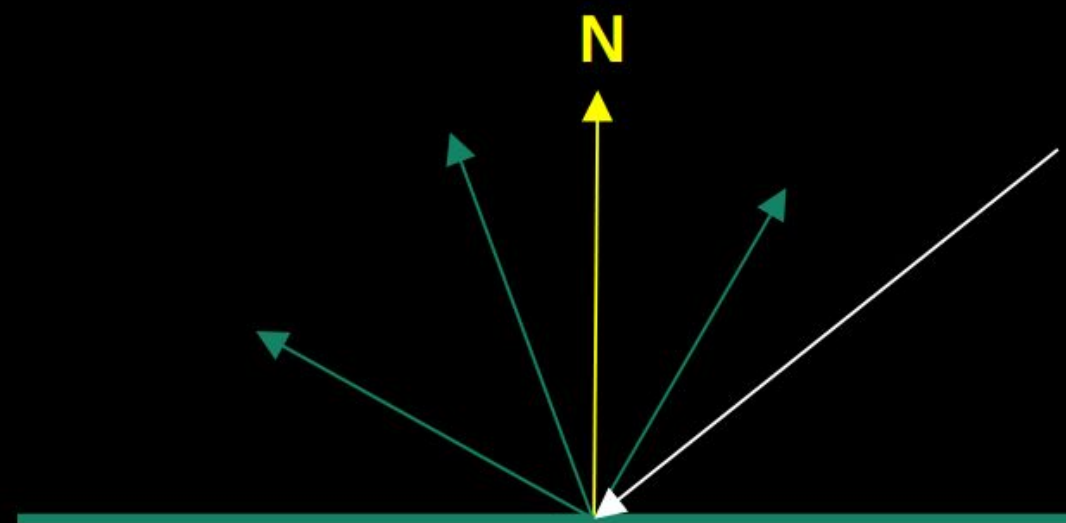
Use a Lambertian BRDF. More complex alternatives often make only a very subtle quality difference.

$$f_{\text{lambert}} = \frac{c}{\pi}$$

Albedo (surface colour)
RGB in [0,1]

Normalization term
(reflectance equation)

Remember that here we are simulating light that penetrates the surface and bounces back (tinted; dielectrics only).



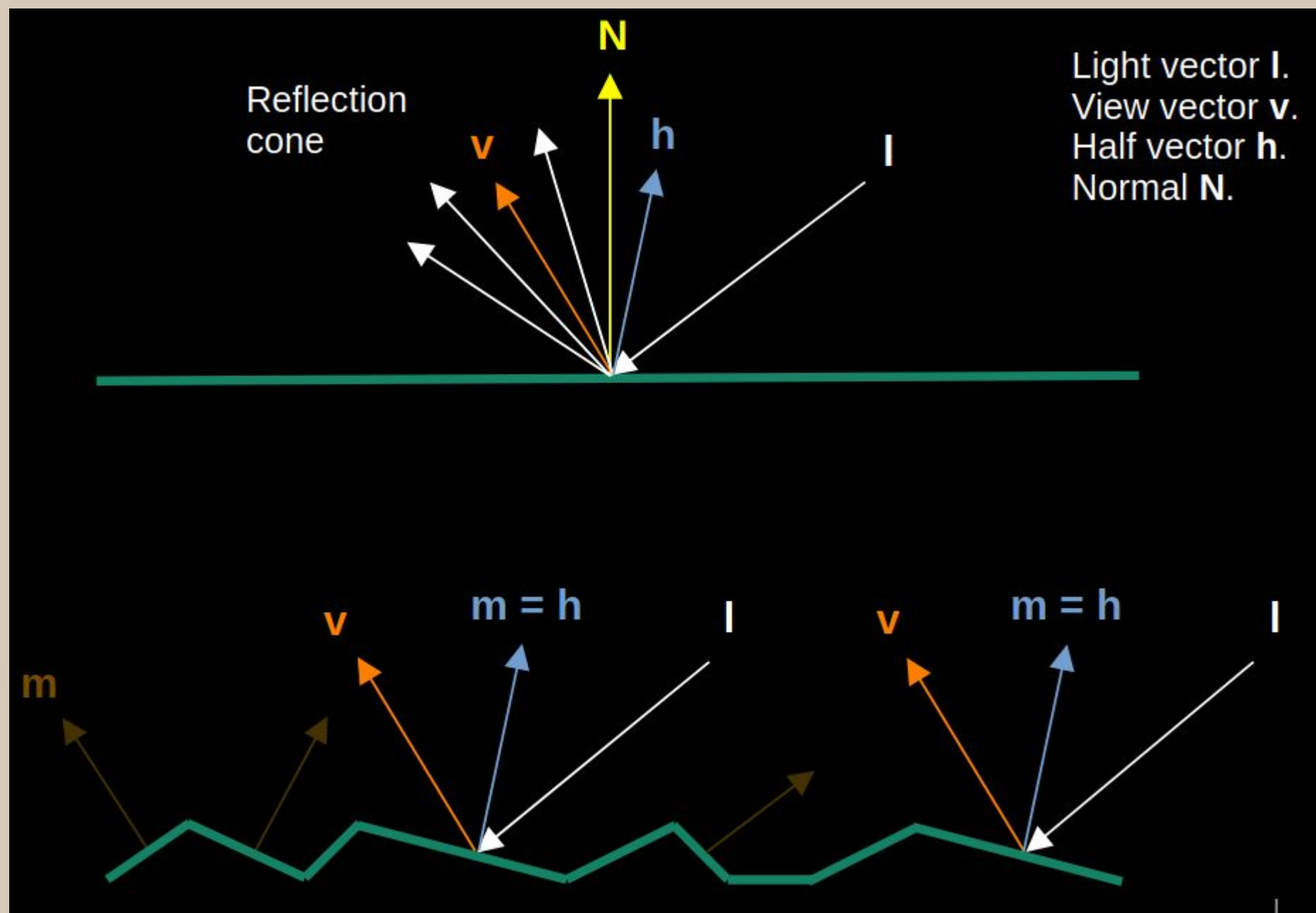
But for the specular term...
we were talking about not perfectly
planar surfaces, weren't we?

Microfacet Theory

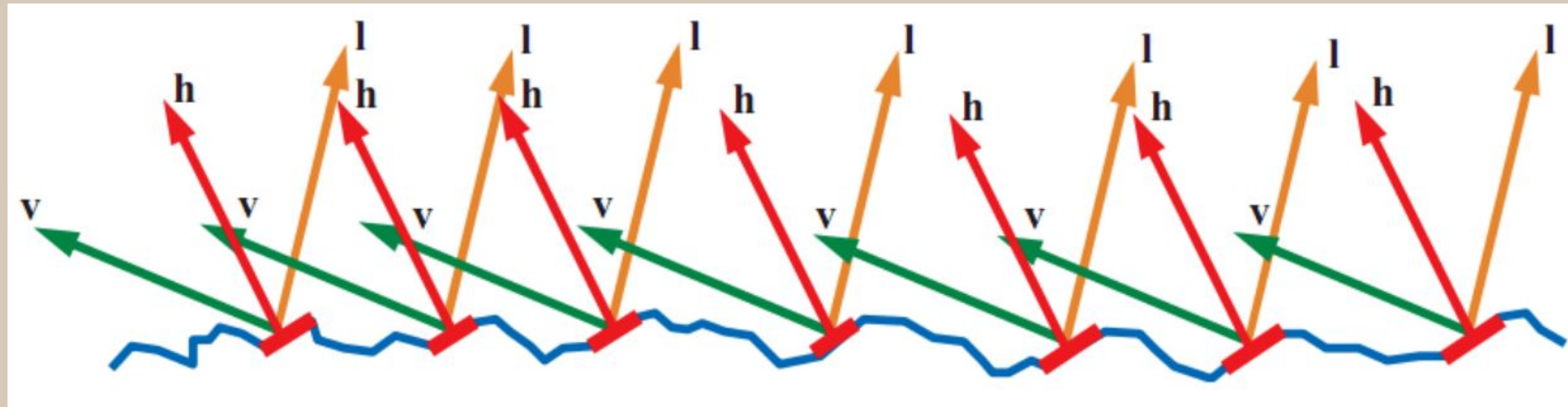
Surface is modeled as a set of microfacets.

Each microfacet is optically flat.

In shading, v and l are given: v is the reflected ray of interest!

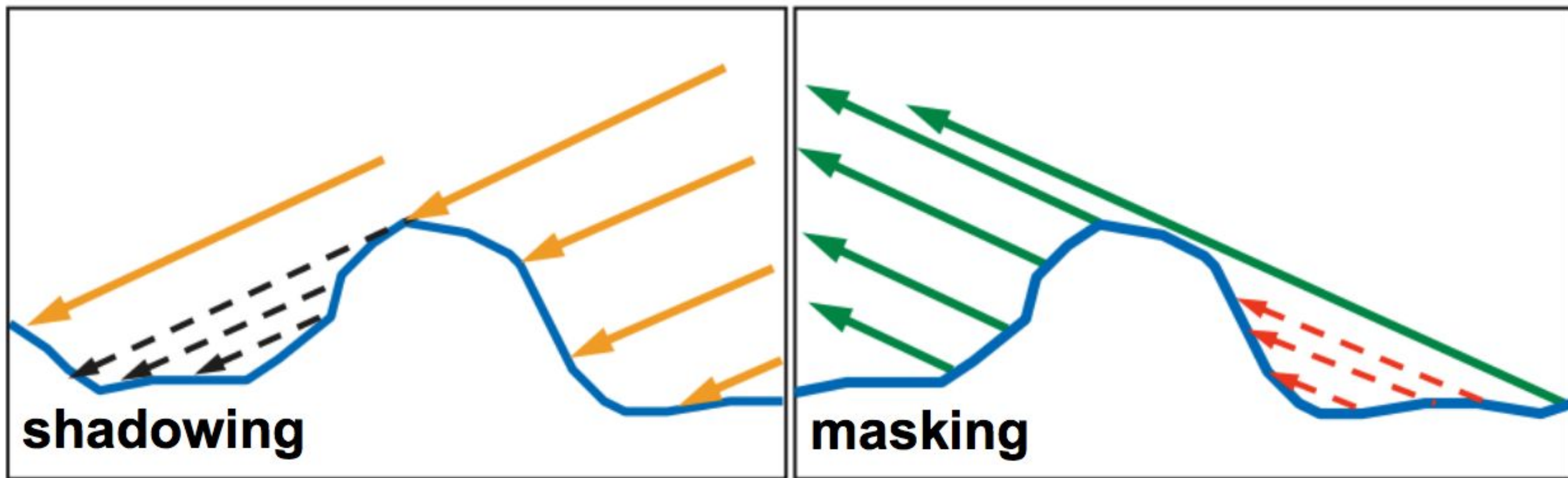


Not all microfacets will reflect light...



Only microfacets with microgeometry normal $\mathbf{m} = \mathbf{h}$ contribute to reflection!

Not all microfacets with $m = h$ will contribute:



But, even not all of them contribute!

Light may be blocked from the direction of l (shadowing) or v (masking).

Inter-reflections not considered!

Microfacet Specular BRDF

Fresnel
Reflectance

Geometry
Function

Microgeometry
NDF

NDF: Surface area of microfacets with microgeometry normal $\mathbf{m} = \mathbf{h}$

G: % of microfacets with $\mathbf{m}=\mathbf{h}$ that are neither shadowed nor masked.

F: % of reflected vs refracted light.

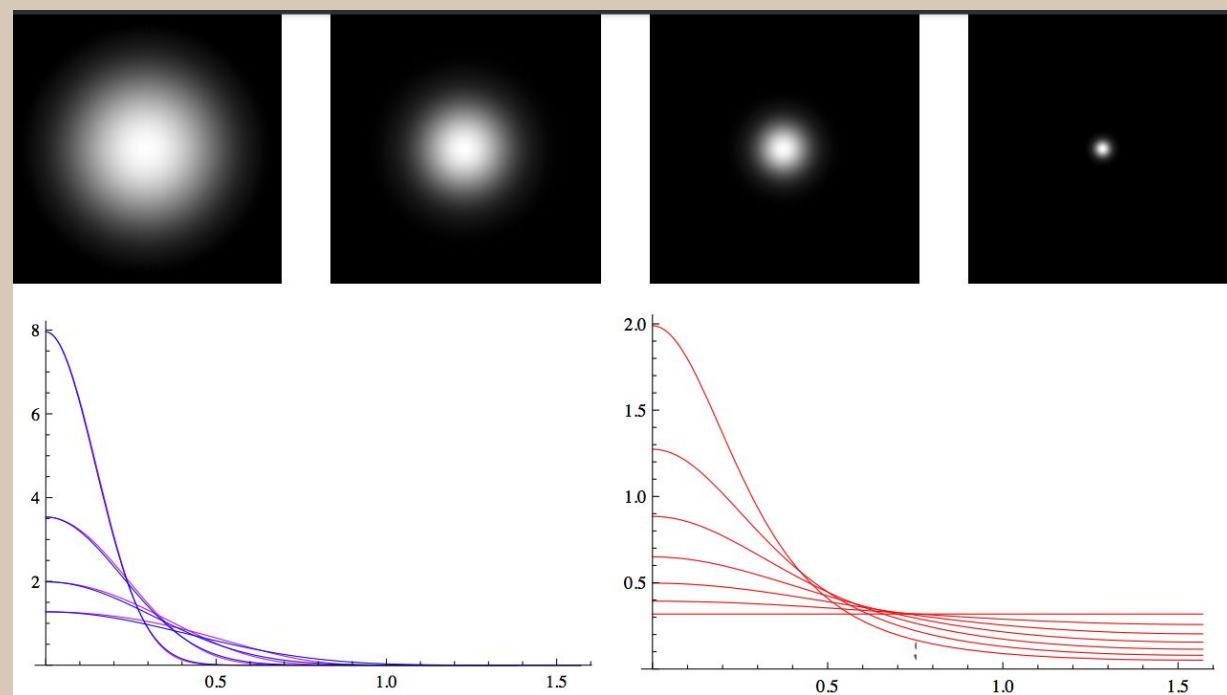
$$f(\mathbf{l}, \mathbf{v}) = \frac{F(\mathbf{l}, \mathbf{h})G(\mathbf{l}, \mathbf{v}, \mathbf{h})D(\mathbf{h})}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}$$

Normalization term for G

Microgeometry NDF - D(h)

i.e. concentration of surface point oriented such that they could reflect light from l to v

Popular choice: Trowbridge-Reitz GGX



$$D(h) = NDF_{GGXTR} = \frac{\alpha^2}{\pi((n \cdot h)^2(\alpha^2 - 1) + 1)^2}$$

Surface roughness

0.1 0.3 0.5 0.8 1.0

Surface roughness

Scalar valued. Determines size, brightness and shape of specular highlight. Usually Gaussian + roughness

Geometry Function - $G(l, v, h)$

i.e. percentage of surface points with $m=h$ that are not shadowed or masked. Essentially BRDF energy conservation

Smith's method with a Schlick GGX is a popular choice:

Source: learnopengl.com



Surface roughness

Does not depend on h , but n .
See references.

$$G(n, v, l, k) = \overset{\text{Masking}}{G_{\text{SchlickGGX}}(n, v, k)} \overset{\text{Shadowing}}{G_{\text{SchlickGGX}}(n, l, k)}$$

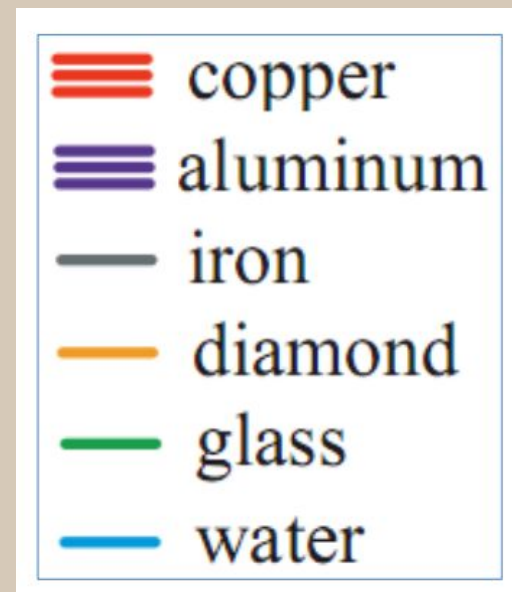
$$G_{\text{SchlickGGX}}(n, v, k) = \frac{n \cdot v}{(n \cdot v)(1 - k) + k}$$

$$k_{\text{direct}} = \frac{(\alpha + 1)^2}{8}$$

$$k_{\text{IBL}} = \frac{\alpha^2}{2}$$

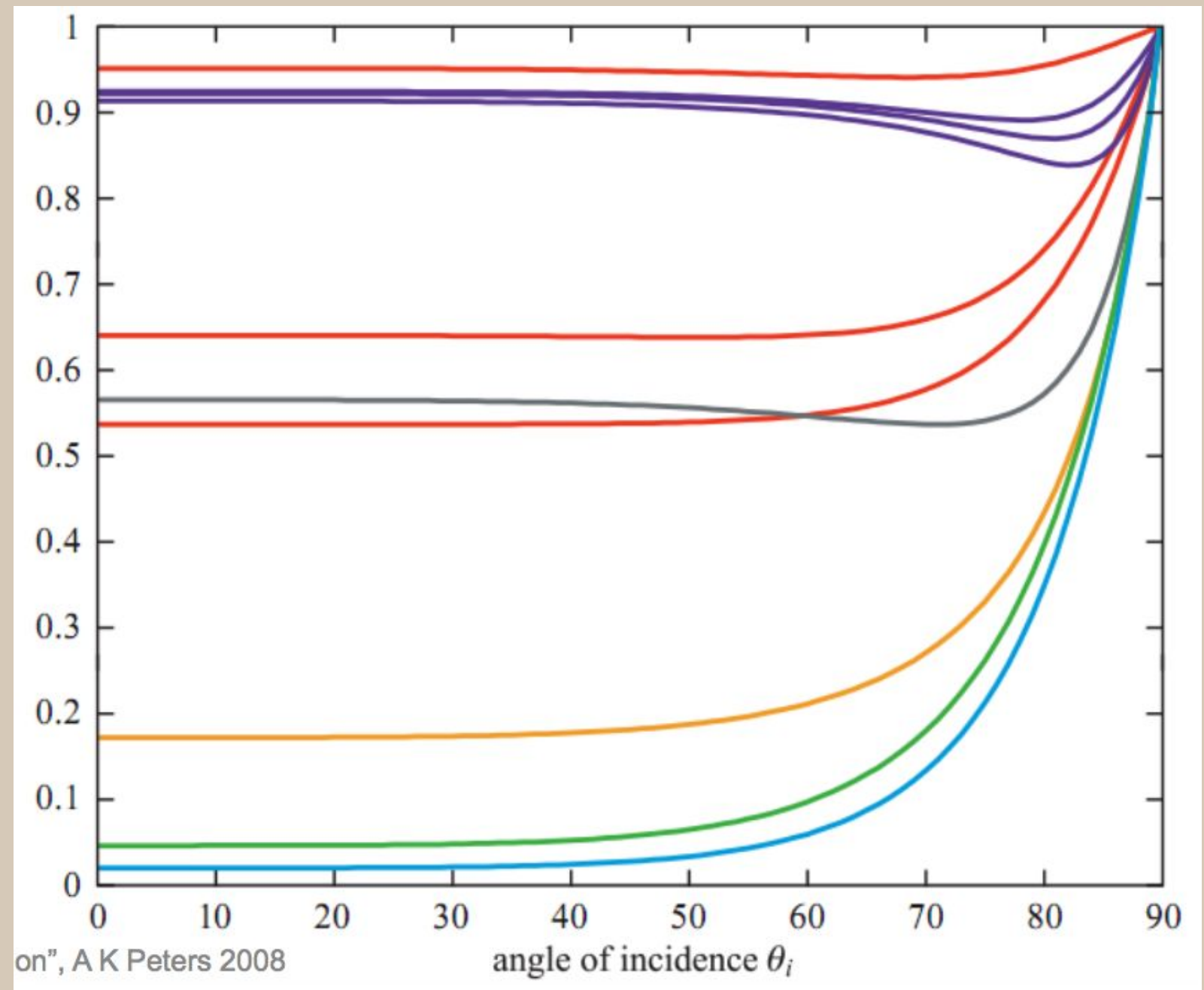
Fresnel Reflectance - $F(i, h)$

i.e. how much of the incoming light is reflected from each of the active surface points



$F_0 = F(0^\circ)$
characteristic
specular color

Complex equations...



The Schlick Approximation to Fresnel

Fairly accurate, cheap, parametrized by F_0

- For microfacet BRDFs ($\mathbf{m} = \mathbf{h}$):

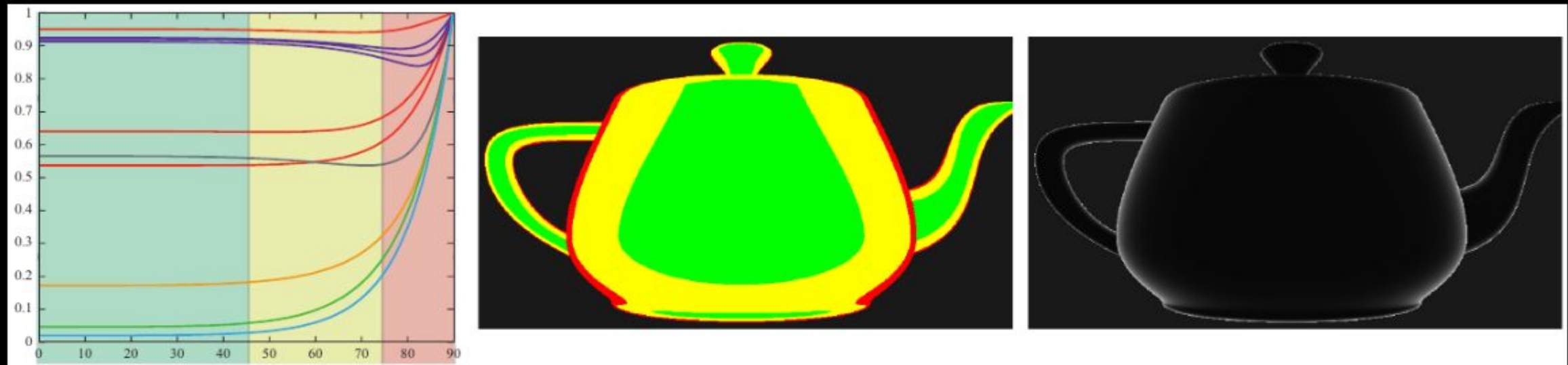
$$F_{\text{Schlick}}(F_0, \mathbf{l}, \mathbf{h}) = F_0 + (1 - F_0)(1 - (\mathbf{l} \cdot \mathbf{h}))^5$$

Approximation based on specular color instead of index of refraction

Reflectance at normal incidence. RGB in $[0,1]$

Angle of incidence

Where will we perceive it?

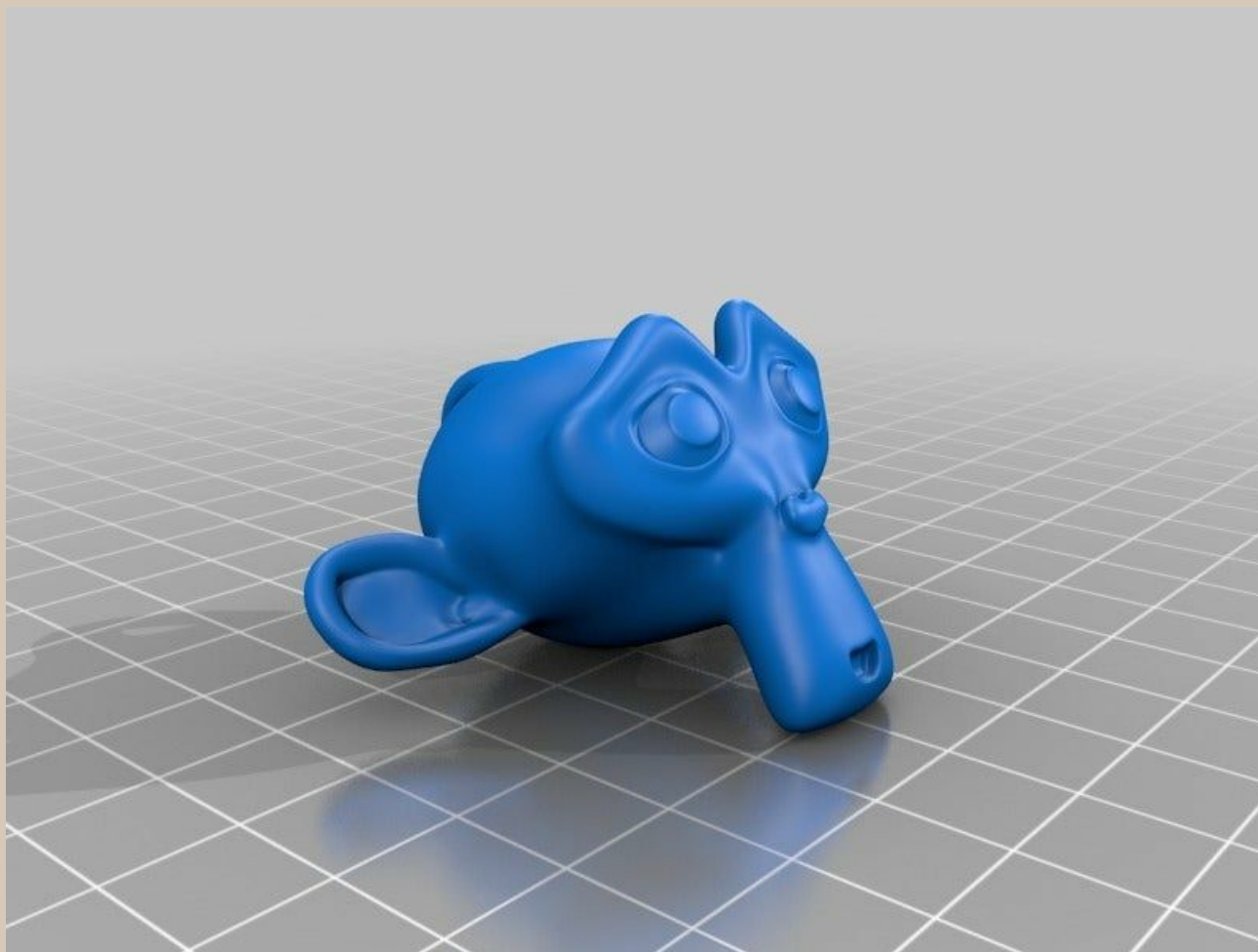


Smooth metallic surface, angle n, v .

Source: PBS course.

At this point...

we are able to render plastic and
metallic monkeys!



Plastic



Metallic



Fast Realistic Rendering

Physically-Based Rendering

Master in Innovation and Research in
Informàtics

Facultat d'Informàtica de Barcelona
Universitat Politècnica de Catalunya
- BarcelonaTech

Imanol Muñoz Pandiella
imanol.munoz@upc.edu



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

ViRVIG



Previously,
on last FRR episode...

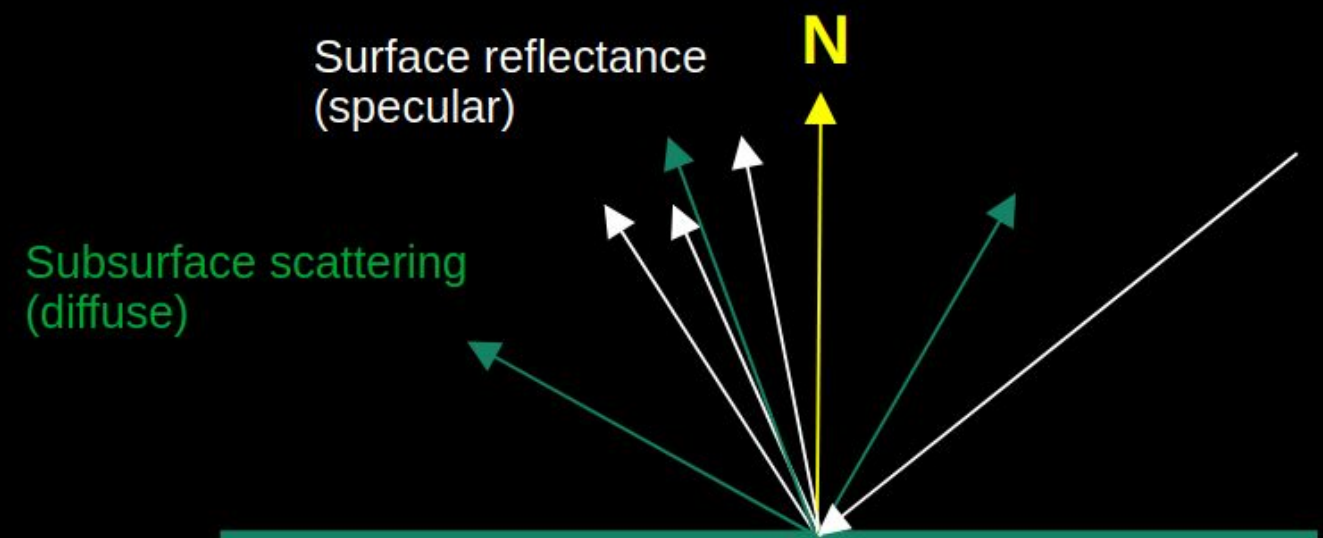
Cook-Torrance BRDF

$$f_r = k_d F_{Lambert} + k_s F_{Cook - Torrance}$$

We will look at the widely-used Cook-Torrance BRDF.

Many alternatives exist resulting from picking different diffuse and specular terms.

Quality-performance trade-off.



Fast? Realistic Rendering

Everything we have seen so far is *for a given light direction*.

N analytical lights → evaluate equations N times.

How should we handle environment lights?

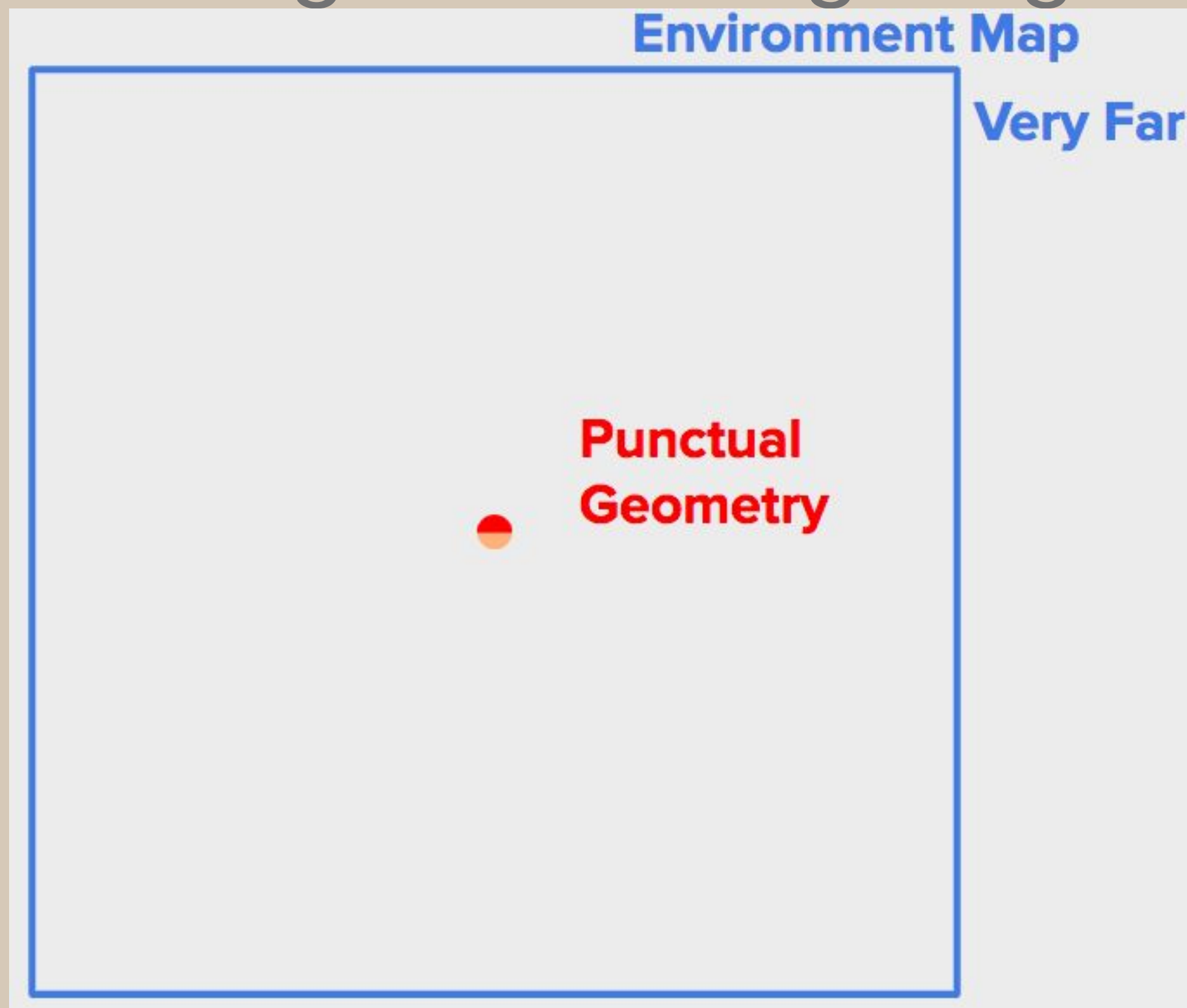
- In principle, sample and evaluate equations N times. (e.g., path tracing)
- Too expensive for real-time rendering.

$$L_o(p, \omega_o) = \int_{\Omega} \left(k_d \frac{c}{\pi} + k_s \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)} \right) L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

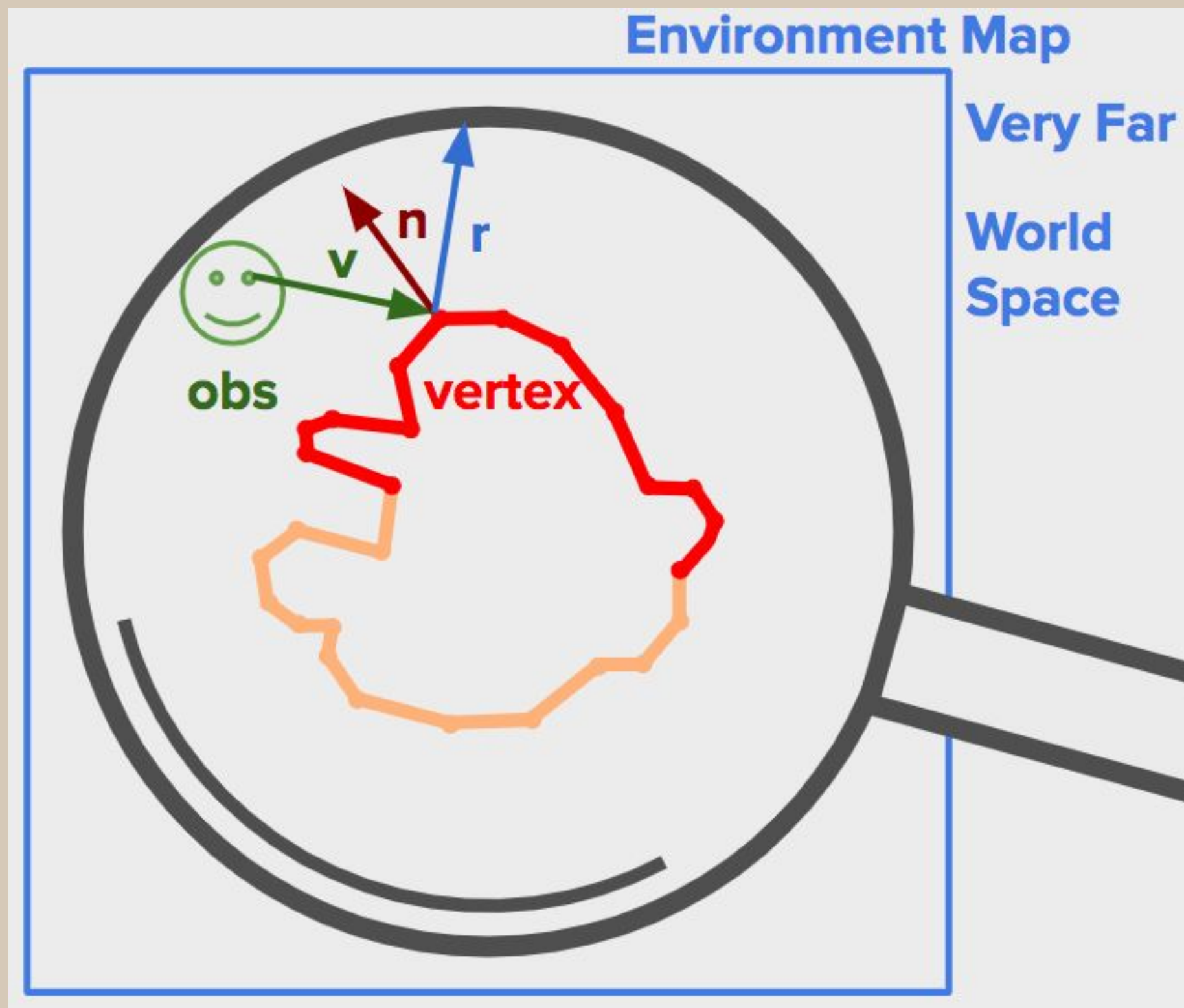
N times

But...
it was a FAST rendering course, isn't
it?

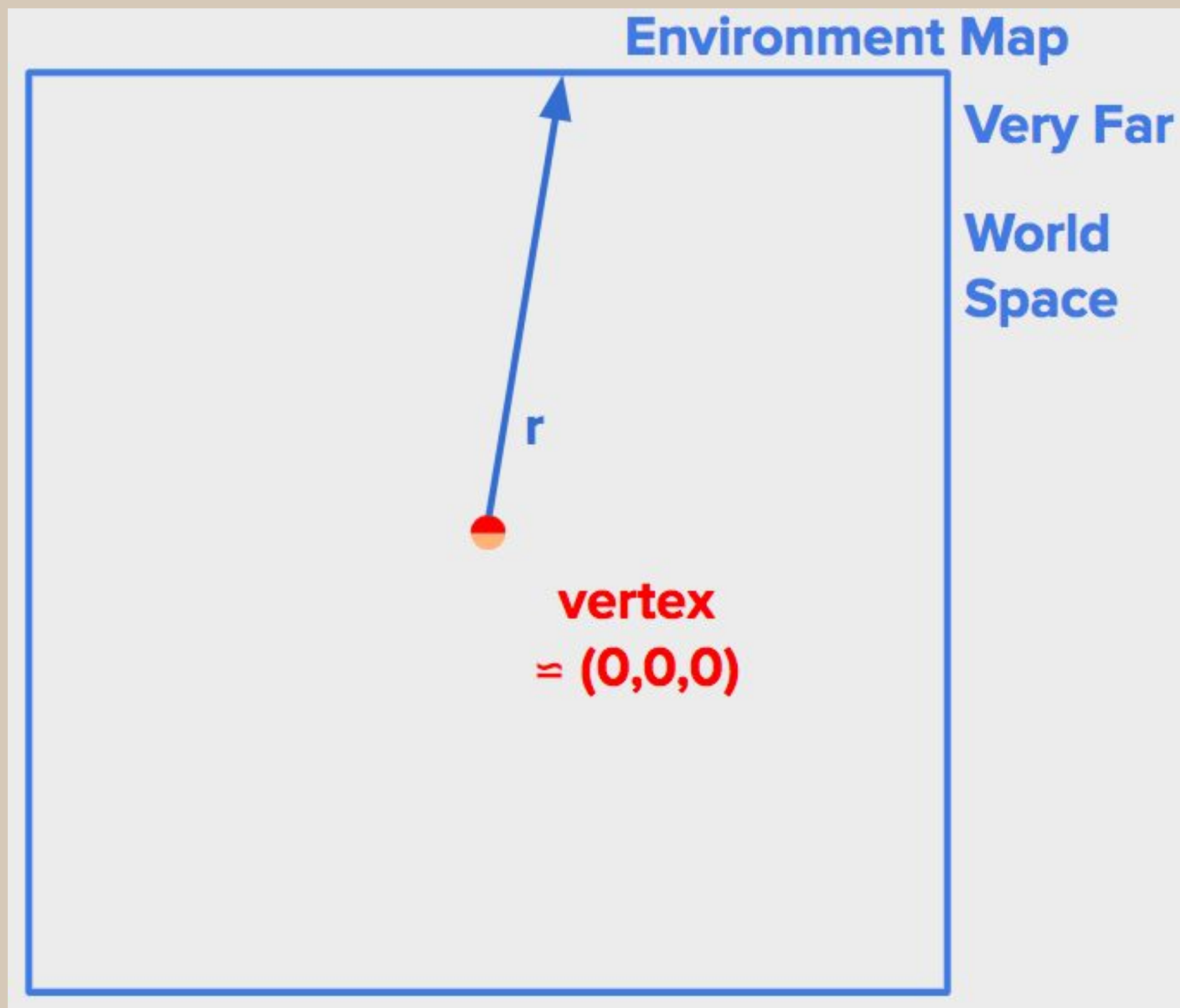
Image Based Lighting



Reflection



Reflection



IBL on PBR

Take the diffuse and specular components apart and handle separately:

$$L_o(p, \omega_o) = \int_{\Omega} \left(k_d \frac{c}{\pi} \right) L_i(p, \omega_i) n \cdot \omega_i d\omega_i + \int_{\Omega} \left(k_s \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)} \right) L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

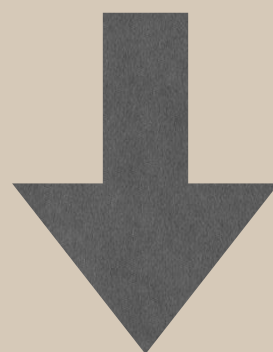
Diffuse IBL

Specular IBL

Diffuse Irrradiance

$$L_o(\mathbf{v}) = \int_{\Omega} f(\mathbf{l}, \mathbf{v}) \otimes L_i(\mathbf{l}) (\mathbf{n} \cdot \mathbf{l}) d\omega_i$$

$$f(l, v) = k_d * f_d + k_s * f_s$$



$$L_o(v) = k_d f_d \int_{\Omega} L_i(l) * (\mathbf{n} \cdot \mathbf{l}) d\omega_i$$

$$f_{\text{Lambert}}(\mathbf{l}, \mathbf{v}) = \frac{\mathbf{c}_{\text{diff}}}{\pi}$$

$\mathbf{c}_{\text{diff}}$: fraction of light reflected, or diffuse color

Diffuse Irrandiance

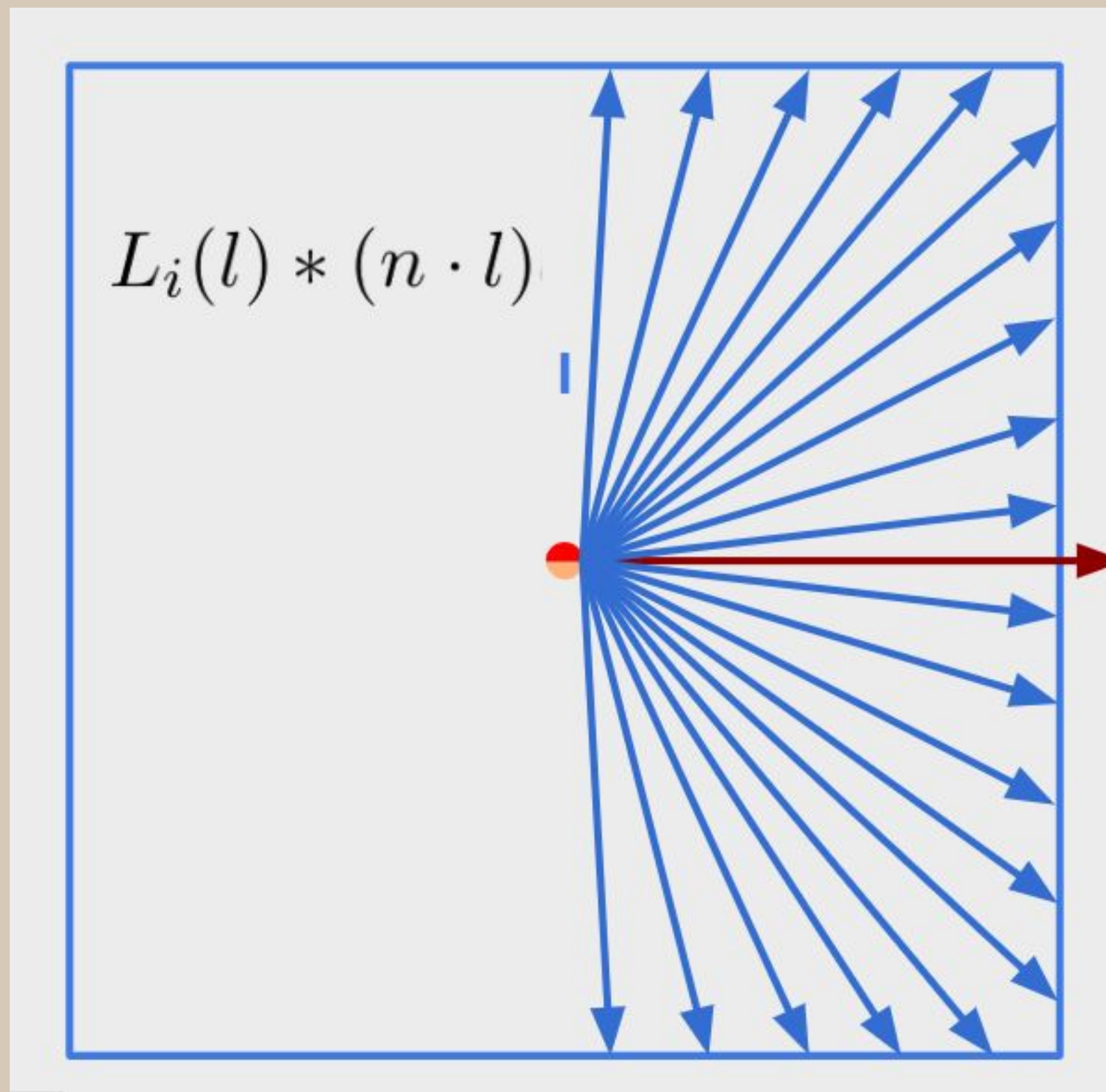
Diffuse BRDF is a constant; does not depend on light or view directions:

$$L_o(p, \omega_o) = k_d \frac{c}{\pi} \int_{\Omega} L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

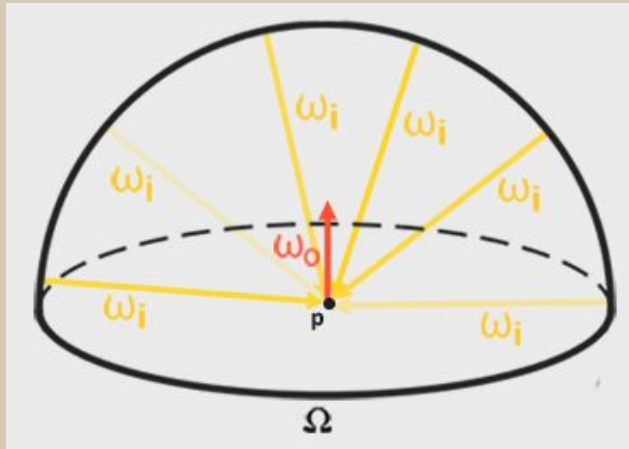
Comes out

Irradiance map

Diffuse Irrandiance

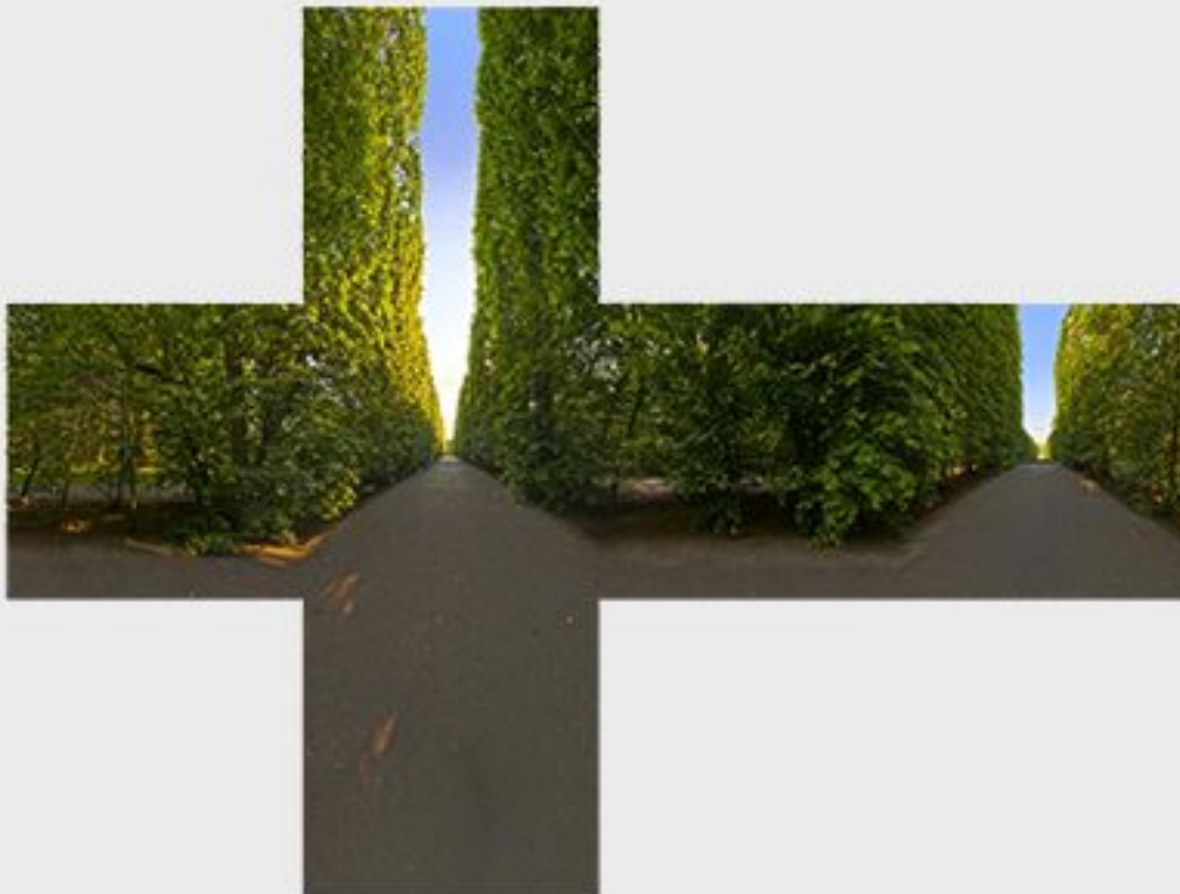


Diffuse Irrandiance



We can pre-compute this convolution and store it in a cube map!

$$L_o(p, \phi_o, \theta_o) = k_d \frac{c\pi}{n_1 n_2} \sum_{\phi=0}^{n_1} \sum_{\theta=0}^{n_2} L_i(p, \phi_i, \theta_i) \cos(\theta) \sin(\theta) d\phi d\theta$$



Specular Irradiance

Specular BRDF depends on both light and view directions;
cannot pre-convolute

$$L_o(p, \omega_o) = \int_{\Omega} k_s \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)} L_i(p, \omega_i) n \cdot \omega_i d\omega_i = \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) n \cdot \omega_i d\omega_i$$

But we'll do it anyway using split-sum approximation:

$$L_o(p, \omega_o) = \underbrace{\int_{\Omega} L_i(p, \omega_i) d\omega_i}_{\text{Pre-filtered environment map}} * \underbrace{\int_{\Omega} f_r(p, \omega_i, \omega_o) n \cdot \omega_i d\omega_i}_{\text{BRDF integration map}}$$

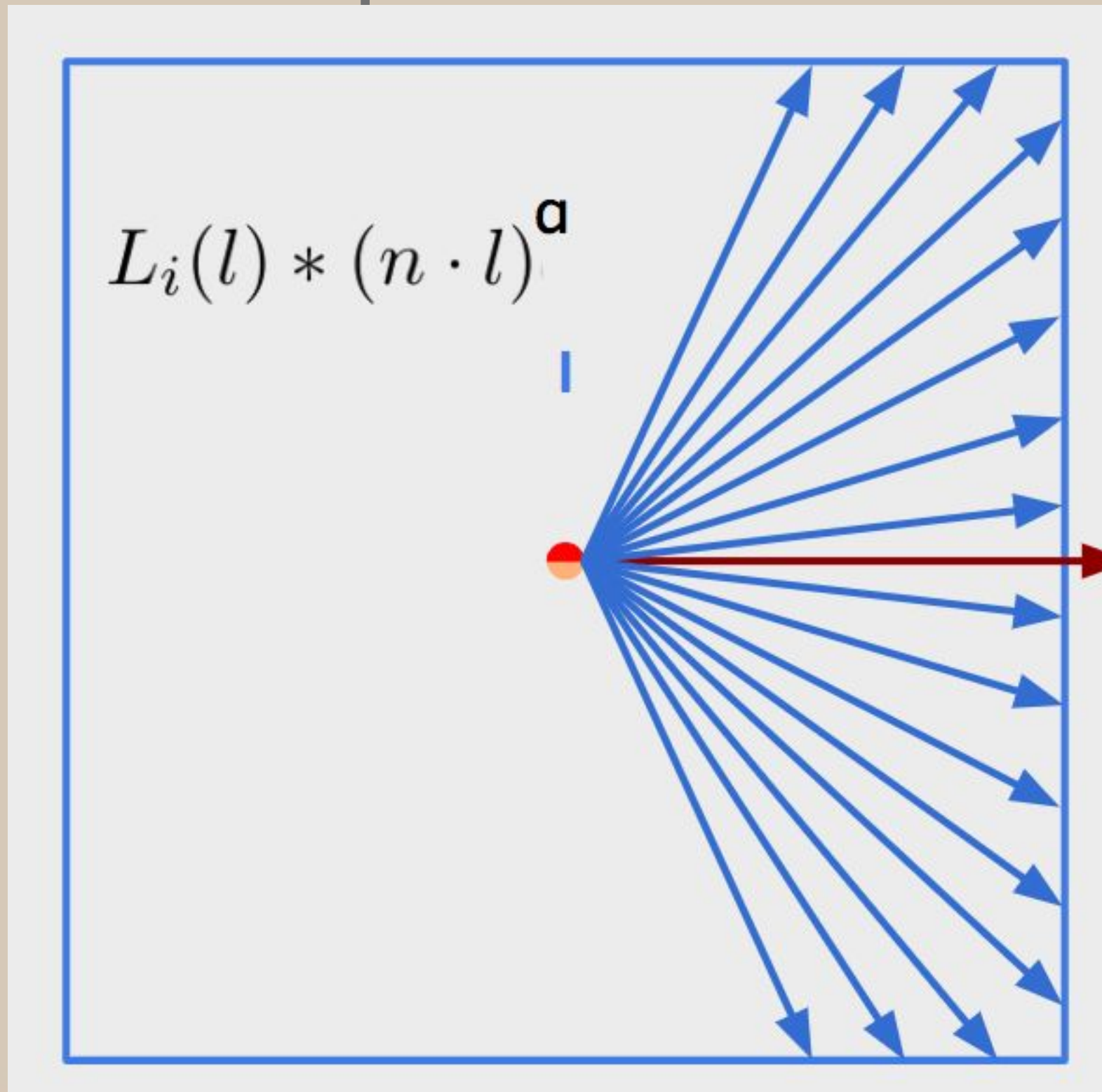
EnvMap considers D and BRDF considers F and G.

Specular Irradiance

$$k_s \int_{\Omega} \frac{FGD}{4(n \cdot l)(n \cdot v)} L_i(l) * (n \cdot l) d\omega_i \approx$$

$$k_s * \frac{FG}{4(n \cdot v)(n \cdot v)} \int_{\Omega} D * (n \cdot l) d\omega_i$$

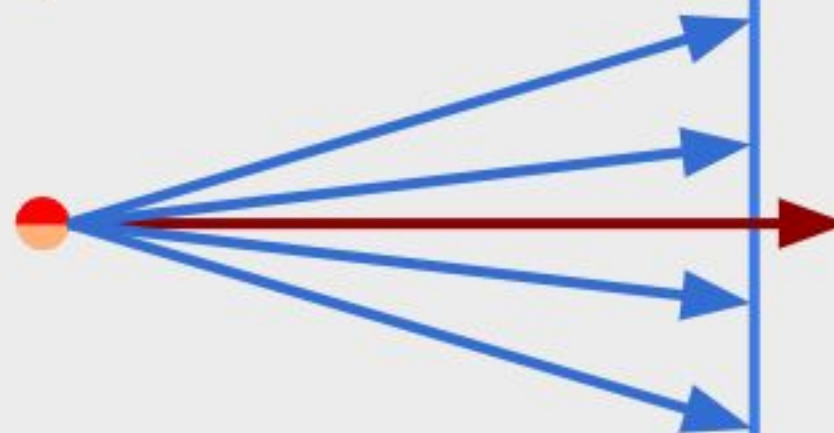
Specular Irradiance - D



roughness ↑↑↑

Specular Irradiance - D

$$L_i(l) * (n \cdot l)^{\alpha}$$



roughness ↓↓↓

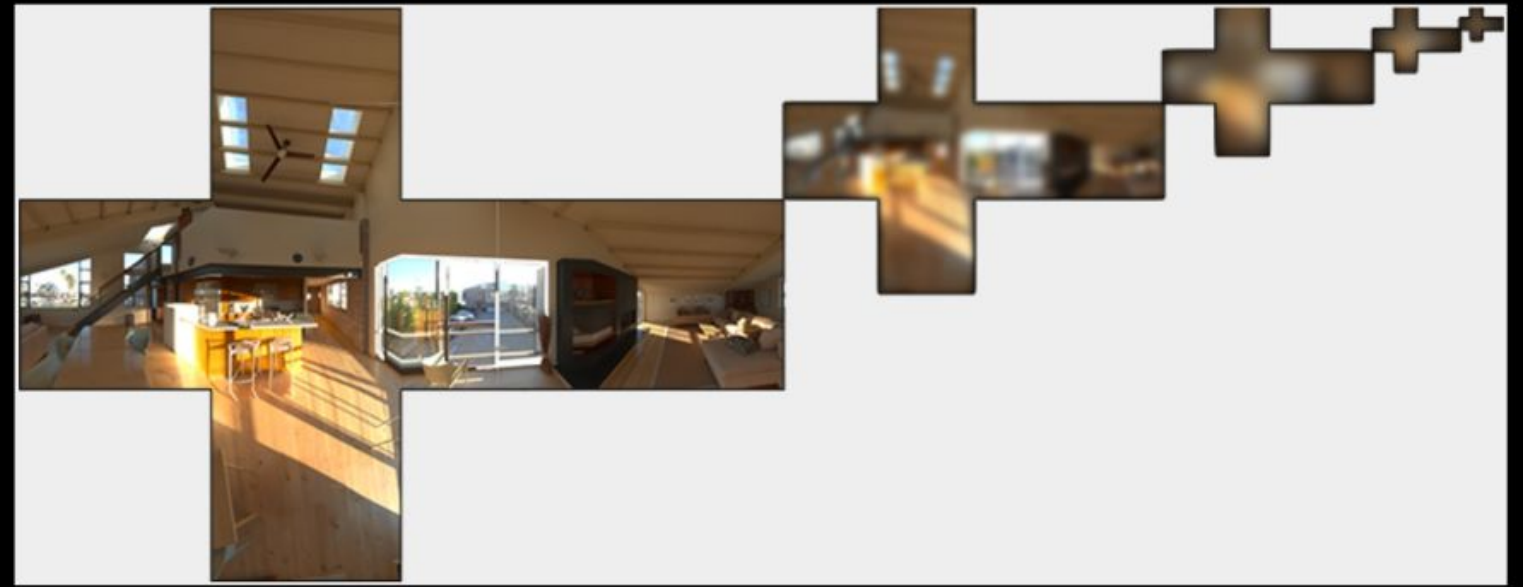
Specular Irradiance - D

The pre-filtered environment map is similar to the irradiance map, but takes roughness into account.

Each mip level encodes the sum of incoming light for cones of a given angle based on surface roughness.

Samples the NDF to generate light directions.

Assumes $w_o = w_i \equiv v = l$ to make the computation feasible given that v is unknown beforehand.



Source: learnopengl.com

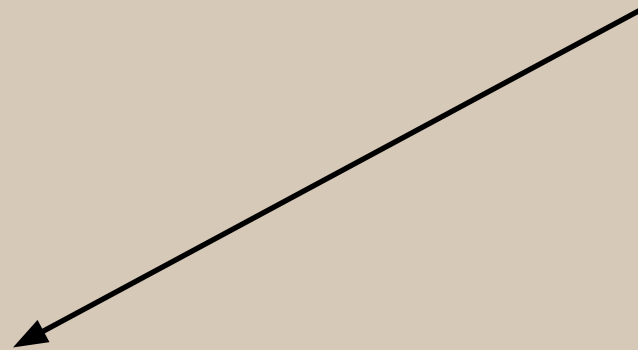
Higher mip $\rightarrow$ rougher surface $\rightarrow$ larger angle.

Specular Irradiance - FG (l = v)

$$k_s * \frac{FG}{4(n \cdot v)(n \cdot v)} \int_{\Omega} D * (n \cdot l) d\omega_i \approx$$

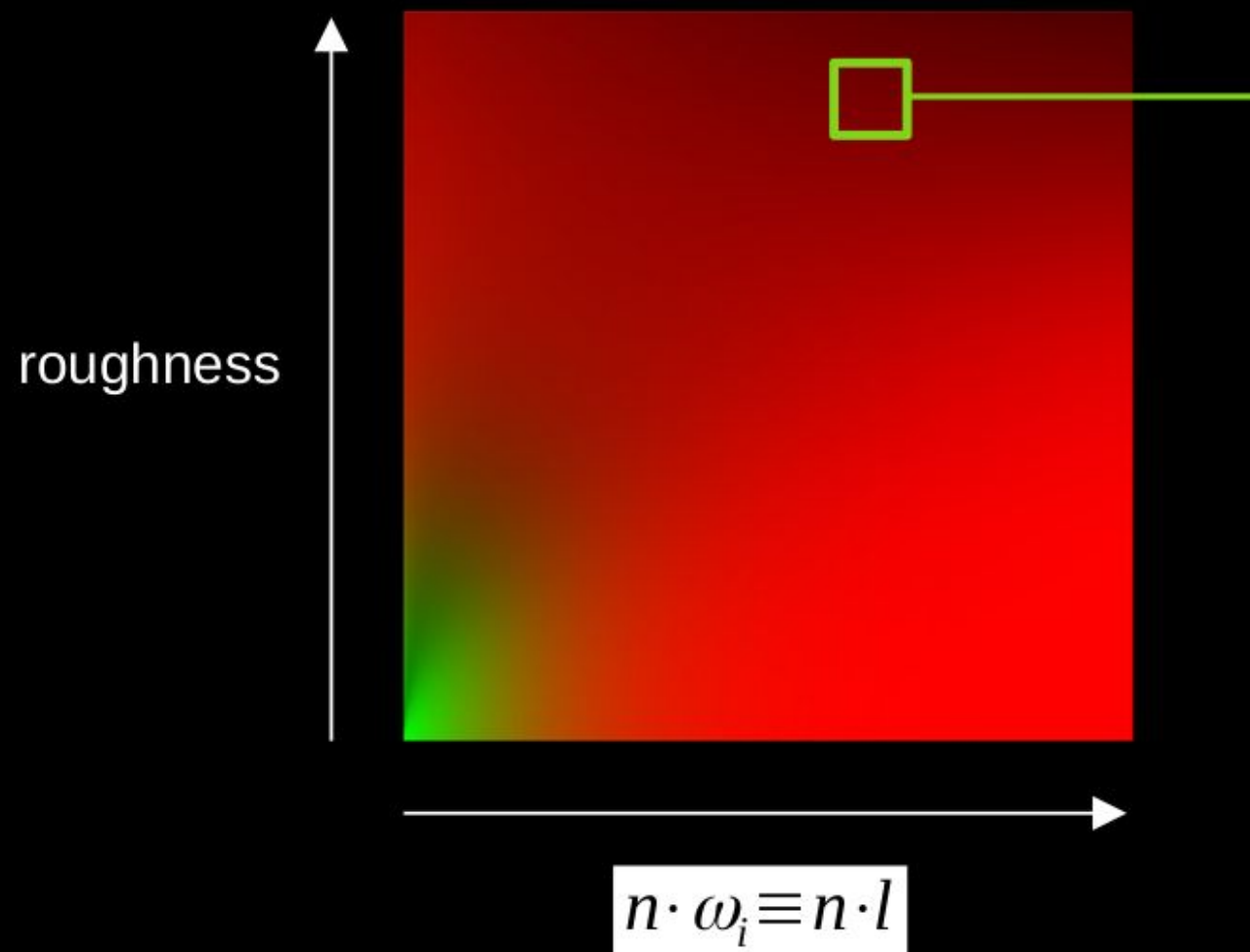
$$k_s * FG' \int_{\Omega} D * (n \cdot l) d\omega_i$$

$$(F_0 + (1 - F_0)(1 - n \cdot v)^5)$$



Alternative: Specular Irradiance - FG

BRDF integration map:



BRDF response given surface roughness and input light direction (light-normal angle).

Red: scale
Green: bias

Scale & bias transform the Fresnel response. Full derivation in *"Real Shading in Unreal Engine 4"* (SIGGRAPH 2013).

So,
after learning PBR and IBL...

Our monkey looks much better!

